PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

School of Engineering

Department of Electronics and Communication Engineering

STUDY of OPTIMIZATION TECHNIQUES and THEIR IMPLEMENTATION ON CNN MODELS FOR IMAGE CLASSIFICATION

A PROJECT REPORT

By

AKSHATA DARJI	20221ECE0113
BHAVANI S MALAGAR	20221ECE0107
AMRUTHA S MALAGAR	20221ECE0106
GNANESH R	20221ECE0098

Under the guidance of,
Ms. Renuka Bhagwat

DECEMBER 2025



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



Department of Electronics and Communication Engineering

BONAFIDE CERTIFICATE

Certified that this project titled "Study of optimization techniques and their Implementation on CNN models for Image Classification" is a bonafide work of "AKSHATA DARJI (20221ECE0113),BHAVANI S MALAGAR (20221ECE0107), AMRUTHA S MALAGAR (20221ECE0106), GNANESH R (20221ECE0098) who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Electronics and Communication Engineering** during 2025-26

Ms. Renuka Bhagwat Project Guide Electronics and Communication Engineering Presidency University	**Dr. Rajiv Ranjan Singh** HOD Electronics and Communication Engineering Presidency University	**Dr. Shrishail Anadinni** Associate Dean School of Engineering Presidency University	**Dr. Abdul Sharief** Dean School of Engineering Presidency University

Examiners

Sl. no.	Name	Signature	Date
1			
2			

DECLARATION

We the students of final year B.Tech in Electronics and Communication Engineering at Presidency University, Bengaluru, named AKSHATA DARJI, BHAVANI S MALAGAR, AMRUTHA S MALAGAR, GNANESH R, hereby declare that the project titled **"STUDY OF OPTIMIZATION TECHNIQUES AND THEIR IMPLEMENTATION ON CNN FOR IMAGE CLASSIFICATION"** has been independently carried out by us and submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

AKSHATA DARJI	20221ECE0113
BHAVANI S MALAGAR	20221ECE0107
AMRUTHA S MALAGAR	20221ECE0106
GNANESH R	20221ECE0098

PLACE: BENGALURU

DATE:

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor**, **Vice Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my integral guide **Ms. Renuka Bhagwat, Assistant Professor** Department of Electronics and Communication Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Rajiv Ranjan Singh, Professor, Head of the Department of Electronics and Communication Engineering**, Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Abdul Sharief**, Dean, **Dr. Shrishail B. Anadinni**, Associate Dean (Core Branches), School of Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to Ms. Swetha G, Project Coordinators, Department of Electronics and Communication Engineering, for facilitating research activities and timely assessments.

We acknowledge Dr. Anilloy Frank for the comprehensive report template, which has been instrumental in enhancing our report's quality.

We are also grateful to Teaching and Non-Teaching staff of Department of Electronics and Communication Engineering and also staff from other departments who have extended their valuable help and cooperation.

AKSHATA DARJI

BHAVANI S MALAGAR

AMRUTHA S MALAGAR

GNANESH R

Abstract

This project explores how different optimization techniques can improve the efficiency and performance of Convolutional Neural Networks (CNNs) used for brain tumor classification from MRI images. Since hardware deployment in real medical environments- such as FPGA boards, edge devices, and low-resource hospital systems-requires lightweight and fast models, optimization becomes essential for practical use.

Eight Convolutional Neural Network (CNN) models were assessed for their ability to classify MRI images into four categories: gliomas, meningiomas, pituitary tumors, and no tumor images; these were: MobileNetV2, ResNet, DenseNet-121, EfficientNetB0, SqueezeNet, FastViT, VGG-16, and ConvNexTV2. Among these eight models, DenseNet-121 had consistently produced the highest level of accuracy throughout all four datasets as it employs dense connectivity which allows it to take advantage of an extensive range of relating features, as well as its unique ability to capture very fine details from the MRI medical imaging source. However, its heavier architecture makes it difficult to deploy on resource-limited hardware platforms.

To address this limitation, multiple optimization strategies-such as batch normalization, weight pruning, layer pruning, network pruning, CLIP-Q quantization, depthwise convolution, tensor decomposition, and knowledge distillation-were implemented on DenseNet-121. To achieve a balance of computational efficiency, parameter reduction, inference speed, and accuracy, these methods were employed. The findings indicate that DenseNet-121, when combined with the Adam optimization algorithm and batch normalization, yielded the highest degree of convergence, stability, and accuracy in classification tasks. Additionally, the usage of techniques like tensor decomposition and knowledge distillation resulted in significant reductions in the model's size while maintaining performance, thus enabling deployment in real-time settings and on hardware with limited resources.

This research demonstrates how optimizing the choice of architecture and the methodology used to optimize a model result in a lightweight, accurate, and hardware-friendly deep learning model. In addition to enhancing the accuracy of classification tasks, the optimized DenseNet-121 model will also improve the practicality and accessibility of medical diagnostic systems through affordability.

Table of Content

Sl. No.	Title	Page No.
	Declaration	3
	Acknowledgement	4
	Abstract	5
	List of Figures	8
	List of Tables	9
1.	Introduction 1.1Background 1.2 Statistics of project 1.3 Prior existing technologies 1.4Proposed approach 1.5 Objectives 1.6 SDGs	 10 10 11 11 13 14
2.	Literature review	15
3.	Methodology	20
4.	Project Management 4.1 Project timeline 4.2 Risk analysis 4.2 Project budget	 24 26 28
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Software Unit Design 5.5 Model Comparison and Selection 5.6 Designing Units 5.7 Standards Used 5.8 Domain model specification	 29 31 32 33 33 34 35 35

	5.9 Communication model	36
	5.10 Functional view	37
	5.11 Other Design aspects	37
6.	Hardware, Software and Simulation	
	6.1 Hardware	38
	6.2 Software development tools	38
	6.3 Software code	39
	6.3.1 Weighted Pruning	44
	6.3.2 Depth-wise Convolution	50
	6.4 Simulation	
	6.5 Summary	50
7.	Evaluation and Results	
	7.1 Test points	51
	7.2 Test plan	53
	7.3 Research Outcomes	66
8.	Social, Legal, Ethical, Sustainability and Safety Aspects	
	8.1 Social aspects	67
	8.2 Legal aspects	67
	8.3 Ethical aspects	67
	8.4 Sustainability aspects	67
	8.5 Safety aspects	67
9.	Conclusion	68
10.	References	70
	Appendix	74

List of Figures

Figure no.	Caption	Page no.
1.1	Proposed Method	13
1.2	Sustainable development goals	14
3.1	The V model methodology	20
3.2	Onion methodology	21
3.3	SDLC phases	22
3.4	Some more methodologies	23
4.1	Project planning timeline	24
4.2	Project Implementation timeline	25
4.3	Risk analysis Flowchart	26
5.1	Functional block diagram	32
5.2	System Flowchart	32
5.3	Domain Model	36
5.4	Communication model suitable for MRI dataset classification	36
5.5	General System Architecture	37

Confusion Matrix and Graphs

7.1	Confusion Matrix of Batch Normalization	54
7.2	Accuracy and Loss of Batch Normalization	54
7.3	Confusion matrix of Weighted pruning	55
7.4	Accuracy and Loss of Weighted Pruning	56
7.5	Confusion matrix of Network pruning	57
7.6	Accuracy and loss of Network Pruning	57
7.7	Confusion matrix of Layer pruning	59
7.8	Accuracy and loss of Layer Pruning	59
7.9	Confusion Matrix of Clip-Q model	60
7.10	Accuracy of Pretrained, QAT and Fine tuning of Clip-Q Model	61
7.11	Confusion matrix of Depth wise Convolution	62
7.12	Accuracy and loss of Depth-wise Convolution	63
7.13	Confusion Matrix of Tensor Decomposition	64
7.14	Accuracy and Loss of Tensor Decomposition	64
7.15	Confusion Matrix of Knowledge Distillation	65
7.16	Accuracy of train and validation of Knowledge Distillation	66

List of Tables

Table no.	Caption	Page no.
2.1	Summary of literature reviews	17
4.1	Example of PESTEL analyses	27
4.2	Project budget	28
5.1	Software system requirements	30
5.2	Comparison of CNN models	33
5.3	Description of Domain model	35
7.1	Comparison of Model before optimization methods	52
7.2	Comparison of Model before optimization methods	52
7.3	Comparison table of DenseNet-121 before and after Batch Normalization optimizer	53
7.4	Comparison table of DenseNet-121 before and after Batch Normalization optimizer	53
7.5	Comparison table of DenseNet-121 before and after Weighted Pruning Optimizer	55
7.6	Comparison table of DenseNet-121 before and after Weighted Pruning Optimizer	55
7.7	Comparison table of DenseNet-121 before and after Weighted Pruning Optimizer	55
7.8	Comparison table of DenseNet-121 before and after Network Pruning Optimizer	56
7.9	Comparison table of DenseNet-121 before and after Network Pruning Optimizer	57
7.10	Comparison table of DenseNet-121 before and after Network Pruning Optimizer	57
7.11	Comparison table of DenseNet-121 before and after Layer Pruning Optimizer	58
7.12	Comparison table of DenseNet-121 before and after Layer Pruning Optimizer	58
7.13	Comparison table of DenseNet-121 before and after Layer Pruning Optimizer	58
7.14	Comparison table of DenseNet-121 before and after Clip-Q Optimizer	60
7.15	Comparison table of DenseNet-121 before and after Clip-Q Optimizer	60
7.16	Comparison table of DenseNet-121 before and after Depth Wise Convolution Optimizer	62
7.17	Comparison table of DenseNet-121 before and after Depth Wise Convolution Optimizer	62
7.18	Comparison table of DenseNet-121 before and after Tensor Decomposition	63
7.19	Comparison table of DenseNet-121 before and after Tensor Decomposition	63
7.20	Comparison table of DenseNet-121 before and after knowledge distillation	65
7.21	Comparison table of DenseNet-121 before and after knowledge distillation	65
7.22	Comparison table of DenseNet-121 before and after knowledge distillation	65

Chapter 1

Introduction

1.1 Background

The brain tumors emerging as a very important health issue since it interferes in the functioning of the brain and in most cases, individuals are not aware when the condition is at early stage. MRI scans are primarily used by the doctors to identify these tumors as the images produced by MRI are much clearer than normal X-rays or CT scans. Although MRI images are useful, the interpretation of these images requires the presence of trained radiologists and the process can be lengthy and, in some cases varies between doctors.

Deep learning has been increasing recently and is applied in numerous medical applications. CNN models are able to pick patterns by looking at images and assist in the recognition of the tumors in a substantially shorter time when compared to manual inspection. MobileNet, ResNet, VGG, EfficientNet, and DenseNet are some of the most popular CNN architecture models. DenseNet-121 was one of the best models in accuracy with our MRI dataset when we tested the models. One issue is that DenseNet-121 is still heavy, that is, it requires more memory and more processing power. Due to this, it is difficult to apply it directly to small clinics or devices with limited hardware.

That is the reason that our project is interested in optimizing DenseNet-121 with the various optimization methods like pruning, batch normalization, quantization, depth wise convolution, tensor decomposition, and knowledge distillation. These techniques assist in reducing the model size, speed and simplify the model to be applied to actual medical conditions.

1.2 Statistics of project

Reported cancer registry of India records that cases of a brain tumor are on the rise. has been growing on a yearly basis. The number of new cases states more than 28,000 each year. The reason why many of these patients fail to receive early diagnosis is due to the absence of specialists, especially in rural areas. Early diagnosis is highly essential as it enhances it probability of successful therapy.

The advanced systems and sufficient number of radiologists are not present in the many hospitals (mostly in small towns). This creates an actual necessity in automated tools to assist the doctors in the quick and correct study of MRI scans. Medical personnel can also be assisted by deep learning models, which minimize the workload of the staff and enhance the process of diagnosing a problem.

These models are however very large and require large scale computers to operate. Accordingly, our project will seek to create a lighter and faster version of DenseNet-121 that will not sacrifice the accuracy significantly. This will allow brain tumor detection devices to be made more affordable to the hospitals that are not able to afford powerful hardware.

1.3 Prior existing technologies

There were previous technologies. A large number of models had been applied in MRI classifications before their optimization techniques.

Prior techniques used:
CNN models like VGG-16 and AlexNet, which were used in older years, were effective but extremely large in size and slow down processing. Better models such as ResNet and Inception are also better performers yet still everything needed good hardware to run. MobileNet and SqueezeNet are lightweight models that are fast, although they do not always provide detailed tumor features. Models that use transformers like FastVit have been experimented as well, but also need powerful GPUs and larger datasets.

Other optimization concepts pruning, quantization, factorization of tensors and distillation of models have also been attempted by different researchers in order to make them faster. However, the vast majority of studies emphasize more on the enhancement of accuracy. Very little literature exists comparing different optimization methods upon same model in order to determine which of them actually works. This is the reason why our project tries to test various optimization methods on a robust model DenseNet-121.

1.4 Proposed Methods

Aim of the project

Our project's major objective would be to create an automated brain tumor classification system based on deep learning method that would be able to recognize the various types of tumor based on MRI image returns with accuracy. This model is designed to aid the doctors in identifying the early presence of the brain tumor particularly in the rural clinic and small hospitals and helps in enhancing the survival of the patient.

Motivation

The brain tumor is life -threatening and requires early detection upon which successful treatment is determined. In most small and rural hospitals where qualified radiologists and equipment is lacking and leads to diagnostic delays. This impelled us to design an affordable solution that will be able to study MRI scans and enhance the survival of a patient.

Plan of Work

Eight CNN models used as the baseline.
Choose DenseNet because it did the most well.
Using several methods of optimization:

- Batch Normalization
- Weight pruning
- Layer pruning
- Network Pruning
- CLIP-Q quantization
- Depth wise separable convolution
- Knowledge distillation
- Tensor decomposition

Comparison of the results with the pre- and post-optimization methodology in terms of accuracy, precision, confusion matrix, total parameters, trainable and non-trainable parameters.

Applications

- Hospital MRI diagnosis support.
- AI-based medical screening.
- Use in rural healthcare centers.
- Portable medical devices.
- Early tumor detection tools.

Limitations

- Needs GPU for training
- Optimization too much may reduce accuracy.
- Dataset variations can affect the performance.

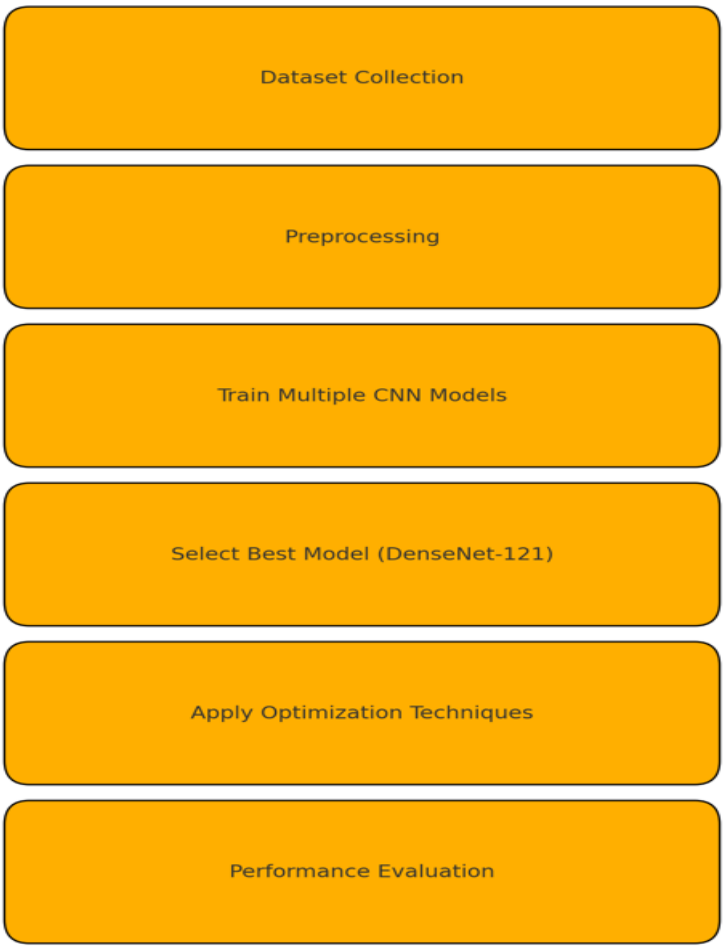


Fig 1.1 Proposed Method

1.5 Objectives

Objective 1 – Behavior
To examine the behavior of DenseNet-121 with the use of alternative optimization methods and its impact on training and validation performance.

Objective 2 – Analysis
The accuracy, confusion matrix, model size, computation time, and the number of parameters of each of the optimized models are to be analyzed and compared**.**

Objective 3 – System Management
An attempt to design a system capable of managing various optimized variants of DenseNet-121, such as training, testing, and model management.

Objective 4 – Security
Patient privacy and integrity of the datasets. To ensure that the MRI images are being manipulated in a safe manner.

1.6 SDGs



Fig 1.2 Sustainable development goals

Our project objectives will align with the UN SDGs:

1. **No Poverty:**

   Our project mainly aims for implementation of automated brain tumor classification system using deep learning techniques so that it is cost efficient and can be helpful in clinical usage where advanced medical diagnosis is very expensive to afford in small hospitals and rural clinics so this system can be helpful in reducing medical costs, and support early detection and treatment which can improve patient survival rates, especially for underprivileged communities. Therefore, the project aligns with the **UN Sustainable Development Goal: No Poverty**, made cost efficient and minimized the burden of low-income patients.

2. **[3] Good Health and well-being:**

   Our project mainly focuses on implementing an automated brain tumor classification using deep learning methods which plays an important role in improving healthcare supports and early detection so that patients can get early treatment. This model analyzes faster and reducing diagnosis time and increasing the chance of successful treatment. By providing an affordable tool which supports small hospitals and rural clinics which contributes in improving the health of many patients and reduces mortality rate. The only goal is to ensure the good health and well being of all the individuals.

Chapter 2

Literature Review

[1] Neamah, K., Mohamed, F., Waheed, S.R., Kurdi, W.H.M., Taha, A.Y. and Kadhim, K.A., 2024.Utilizing deep improved resnet50 for brain tumor classification-based MRI. *IEEE Open Journal of the Computer Society*, *5*, pp.446 456.

This paper enhances ResNet50 through fine-tuning and strong preprocessing to improve tumor classification accuracy on MRI images. The improved model performs better than standard ResNet50 but still requires high computational resources.

[2] Preetha, R., Priyadarsini, M.J.P. and Nisha, J.S., 2024. Automated brain tumor detection from magnetic resonance images using fine-tuned efficientnet-b4 convolutional neural network. *IEEE Access*.

This paper fine-tunes EfficientNet-B4 for MRI-based brain tumor detection and shows that modern lightweight architectures can achieve high accuracy with proper tuning. The model performs strongly with Bayesian hyperparameter optimization and yields reliable results across validation folds.

[3] Wageh, M., Amin, K., Algarni, A.D., Hamad, A.M. and Ibrahim, M., 2024. Brain tumor detection based on deep features concatenation and machine learning classifiers with genetic selection. *IEEE Access*.

The authors merge deep features from multiple CNNs and apply genetic algorithms to select the most important ones, reducing redundancy while improving accuracy. Machine learning classifiers like SVM and XGBoost yield strong performance after feature optimization.

[4] Siddique, A.A., Raza, A., Alshehri, M.S., Alasbali, N. and Abbasi, S.F., 2024. Optimizing tumor classification through transfer learning and particle swarm optimization-driven feature extraction. *IEEE Access*, *12*, pp.85929-85939.

This study combines transfer learning with Particle Swarm Optimization to enhance feature extraction and improve tumor classification accuracy. PSO helps the mode achieve better convergence and avoids poor local minima, outperforming standard optimizers.

[5] Raza, A., Alshehri, M.S., Almakdi, S., Siddique, A.A., Alsulami, M. and Alhaisoni, M., 2024. Enhancing brain tumor classification with transfer learning: Leveraging DenseNet121 for accurate and efficient detection. *International Journal of Imaging Systems and Technology*, *34*(1), p.e22957.

This comparative study shows DenseNet-121 and ResNet50 achieve highest accuracy, while MobileNetV2 offers similar performance with lower complexity. It stresses the importance of efficient models for real-world deployment.

[6] Younis, A., Li, Q., Afzal, Z., Adamu, M.J., Kawuwa, H.B., Hussain, F. and Hussain, H.,
2024. Abnormal brain tumors classification using resnet50 and its comprehensive evaluation. *IEEE Access*, *12*, pp.78843-78853.
ResNet50 is used to classify glioma, meningioma, and pituitary tumors with strong accuracy after augmentation and normalization. High model size is a limitation, suggesting need for optimization.

[7] Rajendran, S., Rajagopal, S.K., Thanarajan, T., Shankar, K., Kumar, S., Alsubaie, N.M., Ishak, M.K. and Mostafa, S.M., 2023. Automated segmentation of brain tumor MRI images using deep learning. *IEEE Access*, *11*, pp.64758-64768.
A hybrid U-Net + 3D-CNN model is proposed for accurate tumor segmentation, capturing both structural and volumetric details. Despite high accuracy, it requires large memory and annotated data.

[8] Panigrahi, S., Adhikary, D.R.D. and Pattanayak, B.K., 2024. Analyzing Activation Functions With Transfer Learning-Based Layer Customization for Improved Brain Tumor Classification. *IEEE Access*.
The study shows Swish activation improves DenseNet-121's performance by capturing subtle MRI features, proving internal layer optimization is effective. Results may vary across datasets.

[9] Lata, K., Singh, P., Saini, S. and Cenkeramaddi, L.R., 2024. Deep learning based brain tumor detection in privacy-preserving smart health care systems. *IEEE Access*.
This paper explores secure brain tumor detection using deep learning while integrating privacy-preserving frameworks. It achieves high accuracy while ensuring patient data confidentiality.

[10] R. Priyanka, G. Swetha, and K. Divya, "A Novel Hybrid System of Detecting Brain
Tumors in MRI."
The authors propose a hybrid feature-extraction and classification system that improves tumor detection accuracy. The method combines multiple techniques to improve reliability.

[11] Alsmadi, H.S., Hamdan, H., Mustapha, N. and Manshor, N., 2025. 3D Convolutional Neural Networks for Brain Tumor Analysis in Multimodal MRI: A Systematic Review. *IEEE Access*.
A survey of 3D-CNN approaches for multimodal MRI analysis, highlighting their advantages in spatial feature extraction but noting their high memory and computational requirements.

[12] Ali, A.A., Hammad, M.T. and Hassan, H.S., 2025. A co-evolutionary genetic algorithm approach to optimizing deep learning for brain tumor classification. *IEEE Access*.
A genetic-algorithm-based optimization method is used to improve deep learning performance in tumor classification. It enhances accuracy but increases training complexity.

2.1 Summary of Literatures reviewed

Sn.	Article Title, Year, Journal name	Methods	Key Features	Metrics	Drawbacks
1	Utilizing Deep Improved ResNet50 for Brain Tumor Classification Based on MRI-2024, IEEE Access.	Transfer learning using improved ResNet50, used image preprocessing and augmentation.	Residual connections which helped the model learn deeper patterns.	Shows higher accuracy than normal ResNet50 and improves stability in predictions.	Model is heavy and slow and needs GPU for training.
2	Automated Segmentation of Brain Tumor MRI Images Using Deep Learning-2023, IEEE Access.	Hybrid model using U-Net and 3D CNN and used GLCM texture features	U-Net finds edges clearly and 3D CNN understands tumor shape in depth.	The model detects tumor boundaries achieves a higher Dice score, and provides better pixel-level classification.	Needs a lot of RAM and GPU power and very slow training. Hard to use without large labelled datasets.
3	Abnormal Brain Tumors Classification Using ResNet50 and Its Comprehensive Evaluation-2024, IEEE Access.	Transfer learning with ResNet50, applied data augmentation with normalization, preprocessing.	Learns deep features through skip connections which works well on multiple tumor types.	Provides high accuracy for all tumor classes and good precision and recall, improves overall F-1 score compared to other models.	Heavy model with many parameters, slow inference on normal computers and needs optimization before clinical use.
4	Enhancing Brain Tumor Classification by a Comprehensive Study on Transfer Learning Techniques and Model Efficiency Using MRI Datasets-2024, IEEE Access.	Compared VGG16, MobileNetV2, DenseNet121, ResNet50,used same MRI dataset for fair testing and applied augmentation	DenseNet and ResNet achieve high accuracy and shows how model efficiency varies across architectures.	Shows performance difference between heavy and light models. DenseNet and ResNet give highest accuracy.	Hard to pick the “best” model because each has trade-offs. Different models need different tuning Some models too heavy for real-time use.

5	Optimizing Tumor Classification Through Transfer Learning and Particle Swarm Optimization-Driven Feature Extraction-2024, IEEE Access	Particle Swarm Optimization (PSO), Inception-based transfer learning and hybrid training with Adam optimizer	PSO helps avoid bad weight initialization. Model finds better weight combinations and improved generalizatio n on small datasets	PSO improves accuracy over normal training, reduces training loss by finding better weights, and gives more stable validation performance.	PSO adds high computation cost. Slower training. Not suitable for very large models.
6	Brain Tumor Detection Based on Deep Features Concatenation and Machine Learning Classifiers with Genetic Selection-2024, Journal of Medical Systems	The study extracted features from multiple CNNs combined them selected the best ones using a Genetic Algorithm and finally classified them using SVM and Random Forest.	It mixes different feature types, uses a genetic algorithm to remove extra features, and helps the classifiers perform better.	It improves the accuracy of SVM, RF, and XGBoost classifiers, reduces model errors through selected features, and makes predictions more reliable across classes.	Very large feature vector. High processing time. Pipeline is long (many steps to complete)
7	Automated Brain Tumor Detection From Magnetic Resonance Images Using Fine-Tuned EfficientNet-B4 CNN-2024, IEEE Access.	Fine-tuned EfficientNet-B4, used Bayesian optimization and applied cross-validation.	It scales efficiently, gives better accuracy than older CNN models, and works well even with small medical datasets.	It achieves higher accuracy than older CNN models improves validation scores through Bayesian tuning, and shows strong cross-validation performance.	Still requires a good GPU. Training time is high. Smaller EfficientNet versions may be better for deployment.

8	Analyzing Activation Functions with Transfer Learning-Based Layer Customization for Improved Brain Tumor Classification-2024, IEEE Access.	They compared activation functions like ReLU, Swish, GELU, and Mish across models such as DenseNet121, VGG16, and MobileNet, and fine-tuned layers to get better output.	Swish gives the best accuracy with DenseNet121 helps capture smoother MRI patterns, and shows that small internal changes can greatly improve performance.	It improves tumor boundary accuracy, gives a higher Dice score, and provides better pixel-level tumor classification.	Activation performance changes for each dataset. Needs trial-and-error to find the best function.

Chapter 3

Methodology

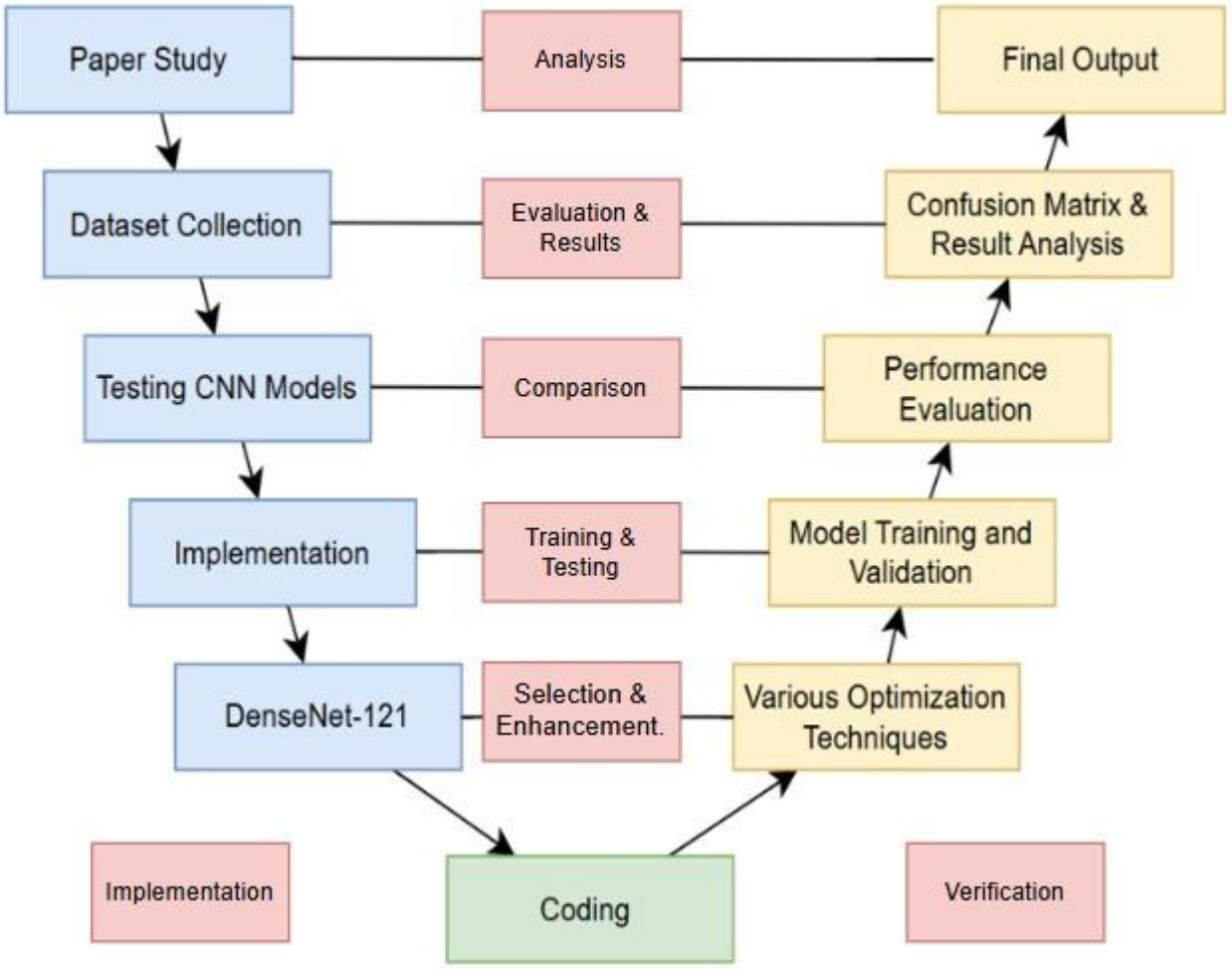


Fig 3.1 The V model methodology

The above diagram shows the overall methodology of our project where it begins from studying the different research papers and collecting the MRI brain tumor datasets. Then we took different CNN models to test and compare among their performance, and DenseNet-121 was selected as the most efficient architecture and 2nd we selected EfficientNet. After implementing the model, various optimization techniques were applied to improve accuracy. The model was trained and validated, followed by performance, evaluation using metrices, confusion matrix, accuracy, loss, and we compared each of the optimization techniques before and after implementing to know that how much accuracy or results changed. Finally, the system produced the classification result for different tumor types.

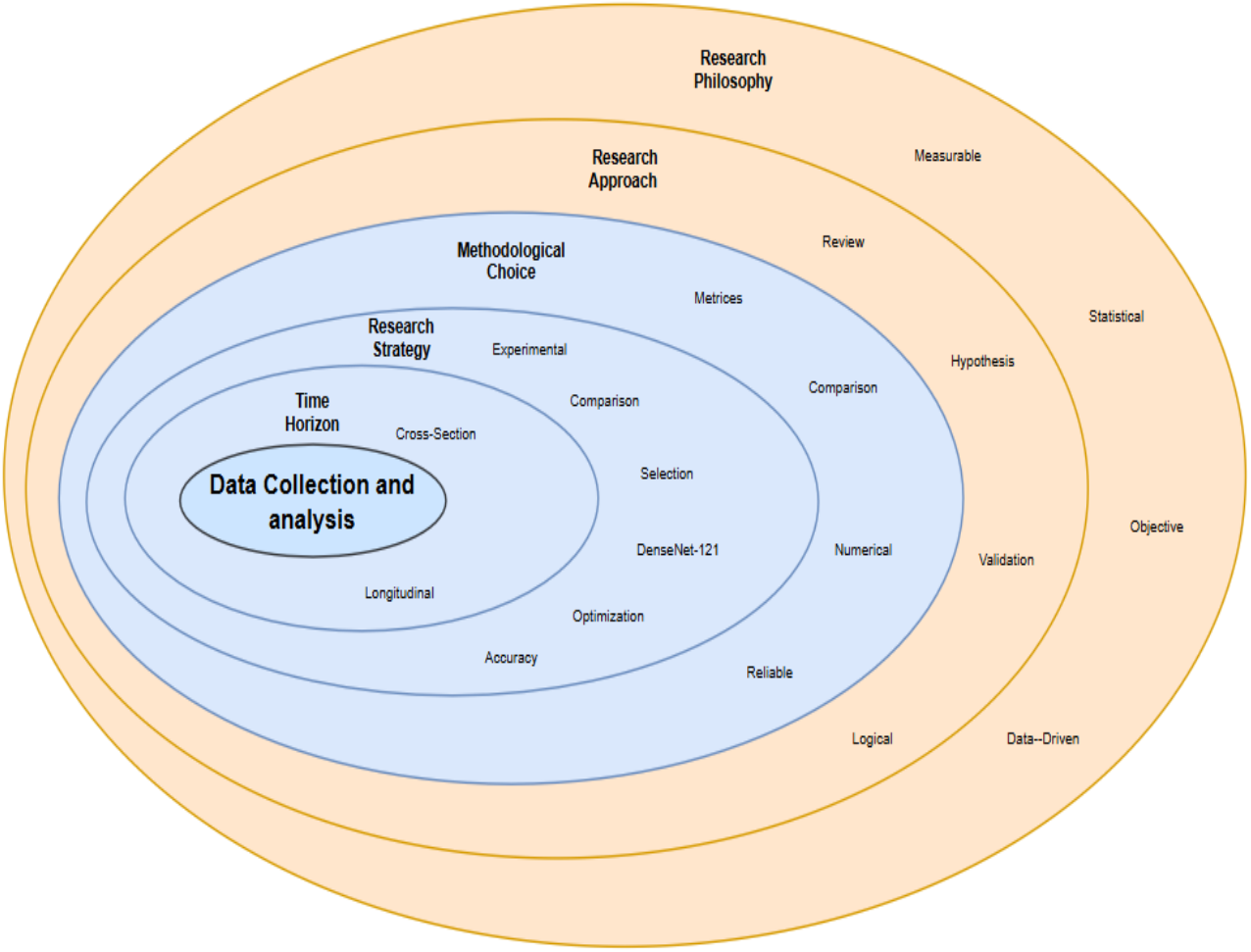


Fig 3.2 Onion methodology

The above diagram shows the onion Methodology. The Outer layer, the **research philosophy** was adopted based on the measurable data, such as accuracy, precision, recall f1 score, rather than assumptions. The **research approach** was used based on the review, validation, of the existing research on brain tumor MRI image classification. The **Methodological choice** was chosen as the project included statistical graph and confusion matrix. The research strategy was used by testing multiple CNN models and selecting DenseNet-121 based on results. A **longitudinal time horizon** was followed, monitoring model performance over several training cycles. And **data collection and analysis** were performed using MRI datasets from open platform to classify tumor types effectively.

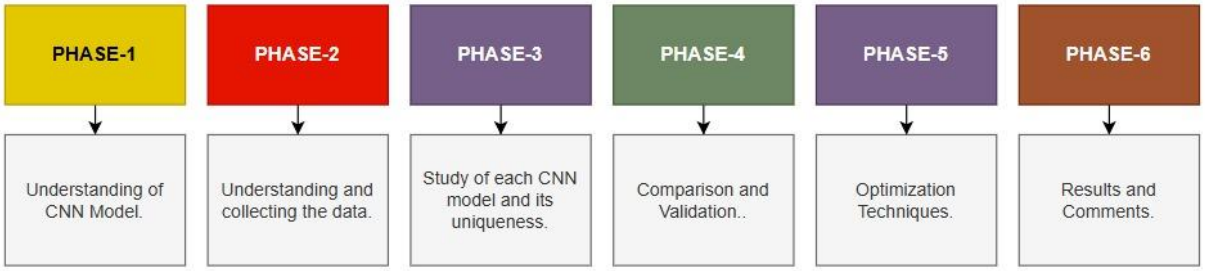


Fig 3.3 SDLC phases

Phase 1: First, we learn how CNN model functions and what is its point and how it functions.

Phase 2: The next step is to gather the all the data that will be required in our project and ensure that it is relevant to our project.

Phase 3: At this stage, we learn the various CNN models and do a comparison and contrast on the distinctive features of each model.

Phase 4: We compare them and test them after that to determine how they perform.

Phase 5: We experiment with alternative optimization strategies to enhance the accuracy, speed and general performance of the model.

Phase 6: Conclusively, we check the results, interpret the results and exchange our findings and thoughts.

PROS	CONS
Helps in early detection of brain tumors, speeding up diagnosis	Sometimes accuracy may vary due to insufficient or imbalanced dataset and variations in MRI scans also affect.
Cost efficient.	Requires GPU for fast training.
Helps rural and small hospitals where experienced radiologists may not be available.	Requires good computing systems, which may not be affordable in small hospitals.
Acts as a reliable second opinion to assist radiologists.	Final Decisions must be made manually.
It can be scaled and improved for further analysis.	Needs more real-world clinical testing.

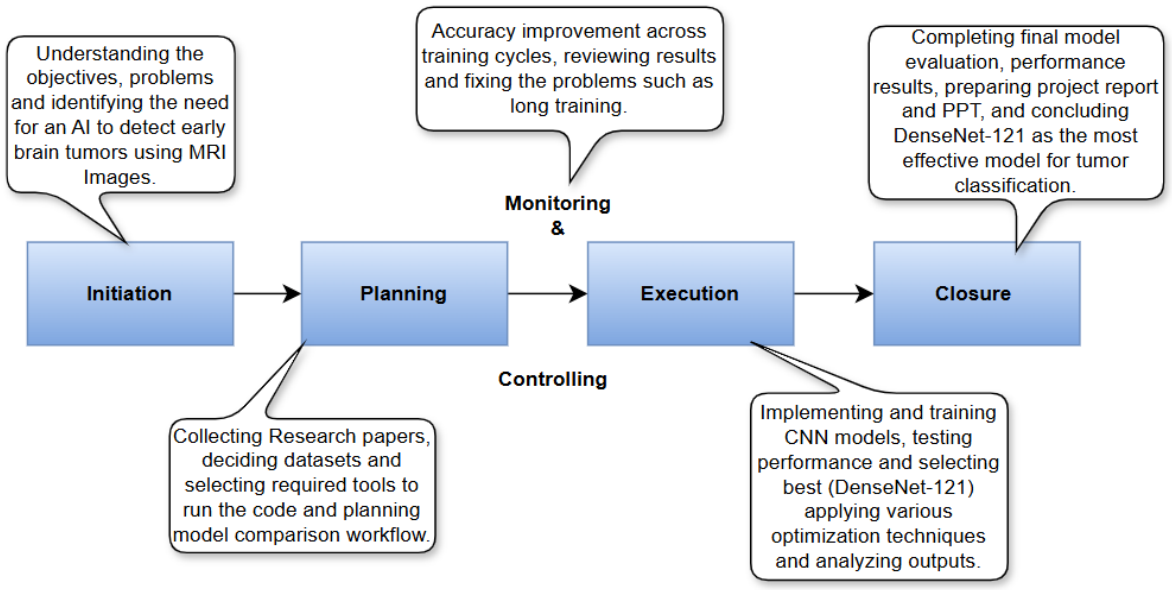


Fig 3.4 Some more methodologies

Chapter 4

Project Management

4.1 Project timeline

To accomplish this project, we have followed a methodical and sequential approach according to an established schedule. In order to maintain proper organization throughout our assignment, we have divided our work into segments, whereby we have assigned each weekly stage of completion, thus preventing any possibility of confusion because of multiple tasks being assigned simultaneously. By assigning a weekly completion of work, we have limited the amount of work that each member of the group was responsible for and therefore made it more controllable.

The first week was spent collecting the MRI brain tumor dataset and understanding the problem. During the second and third weeks, we researched several research papers related to the development of different models of Convolutional Neural Networks (CNNs) and their Optimization techniques. We specifically trained five different types of CNNs: VGG-16, MobileNet, ResNet, EfficientNet and DenseNet-121.

In the subsequent weeks (week 7 through week 9), we utilized several recommended Optimization Techniques on our base CNN model, DenseNet-121. These include Batch Normalization, Pruning, Tensor Decomposition and Knowledge Distillation. The last weeks were used for testing, writing the report, and preparing the final presentation.

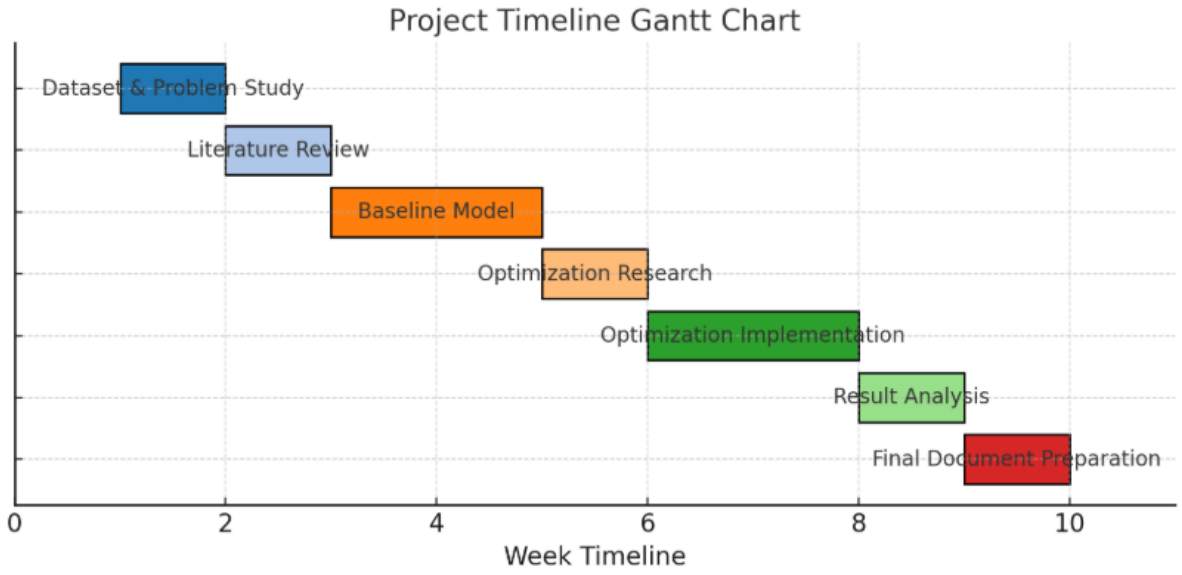


Fig 4.1 Project planning timeline

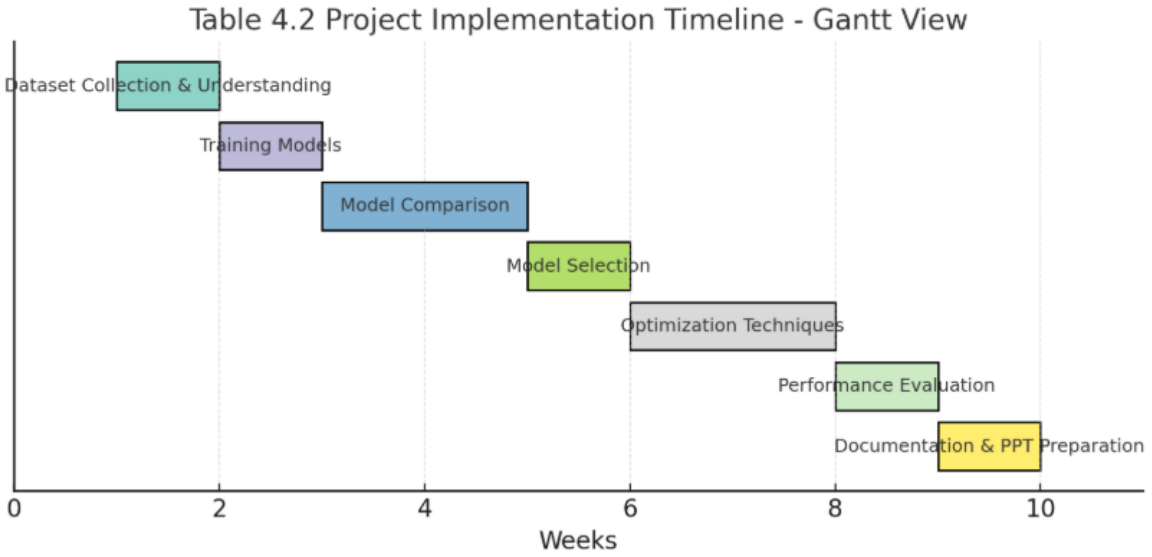


Fig 4.2 Project implementation timeline

The above block diagram shows the step-by-step process followed during the project implementation. We started by collecting and understanding the MRI dataset to get a clear idea of the data format and requirements. We then proceeded to train various deep learning models (VGG-16, MobileNet, ResNet, EfficientNet, and DenseNet-121) in order to compare each of their performances.

Based on all of this information, we decided that DenseNet-121 was the best model due to its improved accuracy when compared with the other models. The following phase involved using various different types of optimization on the DenseNet-121 model to make the model lighter and faster but still retain its high level of accuracy.

Finally, the optimized model was evaluated using performance metrics such as accuracy, loss, and confusion matrix. After getting the final results, documentation and PPT preparation were completed for report submission and presentation.

4.2 Risk analysis

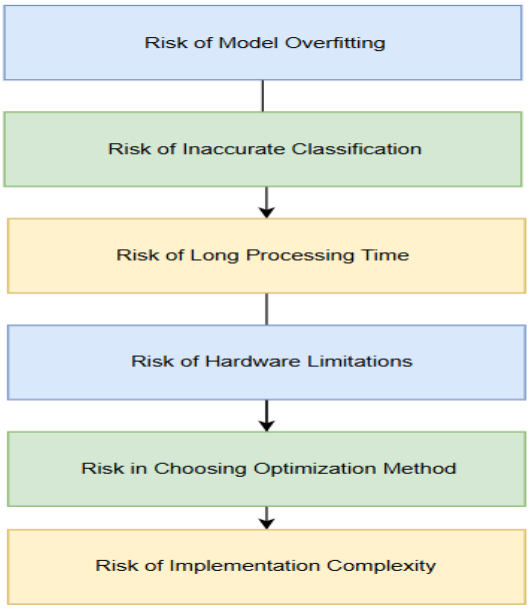


Fig 4.3 Risk analyses Flowchart

- **Risk of Model Overfitting:**
  Since medical datasets are often limited, the model initially showed signs of overfitting, where training accuracy was high but validation accuracy fluctuated. This required careful tuning and the use of optimization techniques to stabilize performance.
- **Risk of Inaccurate Classification:**
  Working with four tumor categories increased the chance of misclassification, especially between visually similar glioma and meningioma cases. This risk needed repeated evaluation with confusion matrices and performance metrics.
- **Risk of Long Processing Time:**
  Deep learning models, particularly DenseNet-121, required lengthy executions times which resulted in slower experimentation and made it more challenging to assess the different variations quickly.
- **Risk of Hardware Limitations:**
  Limited GPU availability caused several training interruptions and failures, forcing reliance on shared cloud resources, which affected the project schedule.
- **Risk of Choosing the Wrong Optimization Method:**
  Each optimization method impacted the model differently, and selecting one without proper evaluation could reduce accuracy. Careful comparison after each optimization was needed.
- **Risk of Implementation Complexity:**
  Integrating techniques like pruning, depth-wise convolution, tensor decomposition, and knowledge distillation required significant technical understanding, and errors during implementation caused delays.

POLITICAL P	ECONOMIC E	SOCIETAL S	TECHNOLOGICAL T	ENVIRONMENTAL E	LEGAL L
-	-Reduce medical costs. -Affordable for small and rural hospitals	-Early detection improves patient health and supports rural hospitals. -Helpful in rural Healthcare.	-Deep Learning, CNN optimization, GPU computing. - medical imaging.	-	- Patient data privacy. -medical AI approval rules.

Table 4.1 Example of PESTEL analysis

PESTEL analysis allows us to know a number of external forces that affect the design and execution of the system. Economically, the solution has the potential to lower the medical bills, and enable advanced healthcare to be affordable, particularly to small and rural hospitals. The effects on society are greater, as the early diagnosis of the disease enhances the results of the patient and promotes healthcare services in rural areas. Technologically, progress has been made and made in the areas of deep learning, CNN optimization, GPU computing, and medical imaging to make efficient and accurate diagnostic systems. Legal angle addresses the matters of data privacy of patients and adherence to medical AI acceptance and regulations. All these factors lead to safe, proper, and responsible use of the technology.

4.3 Project budget

Sl. No	Item Description	Quantity	Approx. Cost (₹)	Total (₹)
1	Laptop / GPU computing usage (Cloud – Google Colab Pro / Kaggle / AWS)	3 months	2,500 / month	7,500
2	Internet and electricity usage	Project duration	500 / month	1,500
3	Software tools (Python, TensorFlow)	—	2,000	2,000
4	Printing, binding, and documentation	1 Reports	1000	1,000
5	Reference materials (journals, papers, books)	—	600	2,400
6	Storage devices (USB drive / Hard disk)	1	1,500	1,500
7	Miscellaneous expenses	—	—	900

Table 4.2 Project budget

The project budget was planned to cover the basic resources we needed throughout the development process. Most of the cost went toward cloud computing support, since training deep learning models requires GPU access for several weeks. We also included regular internet and electricity usage, as well as expenses for printing and binding our project reports. Additional costs were allocated for software tools, research papers, reference material and storage devices to securely manage the dataset and project files. A small amount was reserved for unexpected or miscellaneous needs that often come up during project work. Overall the budget was kept minimal and practical, making sure that every expense supported the real requirements of completing the project smoothly.

Chapter 5

Analysis and Design

- An intelligent computational system development involves analysis and design. These steps allow designing a deep learning solution to a medical diagnostic challenge in the real world in this project. Analysis aims to comprehend the roles that the system will perform, its purpose, data behavior, operational limitations and the results they are supposed to provide. Design is concerned with the manner in which the system achieves these goals- its architecture, computational flow, software modules and methodology of implementation.
- This chapter includes a breakdown of the system requirements, system architecture, block diagram, flow diagram, model selection criteria, unit-wise design, standard, functional, and operational view, and all the critical design considerations that would be required to implement the proposed automated brain tumor classification system through deep learning.

5.1 Requirements

The purpose, behavior, performance expectations, constraints, and resources required to achieve reliable medical image classification are determined by the requirements of the system.

5.1.1 System Purpose

The system aims at automatically labeling images of brain MRI into four classes namely Glioma, Meningioma, Pituitary Tumor, and No Tumor on a deep learning model, supported by DenseNet121. The system is designed to help radiologists because it would give quick, precise, and reliable forecasts. System Behavioral Requirements.

5.1.2. System Behavioral Requirements

- The system should take MRI images as an input.
- Images have to be preprocessed (resized, normalized, generated into tensors).
- The model will have to produce a prediction and class probability scores.
- The working process should be conducive to training, proving, and testing.
- The system has to produce performance indicators like accuracy, loss curve and confusion matrices.
- The model should be able to deal with invisible MRI images when doing inference.

5.1.3 Hardware System Requirements.

Requirement Description-

- **CPU** - Processor Intel i5/i7 or AMD equivalent.
- **GPU** - NVIDIA GPU, which is CUDA capable (recommended).
- **RAM** - Minimum 8 GB (16GB is suggested to train)
- **Storage** - Dataset, logs and model checkpoints, at least 20 GB storage.
- **Display** - Visual level display of graphs.

5.1.4 Software System Requirements.

Software Component	Purpose
1. Python 3.8+	General purpose programmer environment.
2. PyTorch	Model building, training, inference.
3. Matplotlib	Drawing accurate/loss graphs.
4. scikit-learn	Confusion matrices and metrics generation.
5. Jupyter Notebook	Development environment.
6. CUDA Toolkit	Accelerated training.

Table 5.1 Software system requirements

5.1.5 Data Collection Requirements

- MRI imaging should be of high resolution and correctly labeled.
- The data should have four classes namely Glioma, Meningioma, No Tumor, and Pituitary Tumor.
- The photos should be in class specific folders.
- Dataset splits must include:
  - Training set (~70%)
  - Validation set (~15%)
  - Testing set (~15%)
- Patient identity should be safeguarded by anonymizing dataset.

5.1.6 Data Analysis Requirements

- All of the images should be reduced to 224 x 224 pixels in order to fit the input of DenseNet.
- ImageNet standard deviation and ImageNet mean have to be normalized.
- Training images should be augmented by data to ensure that there is a decrease in overfitting.
- Measures to gauge the performance of the model should contain:
  - Accuracy
  - Precision
  - Recall
  - F1-score
  - Confusion matrix
- Learning curves must be interpolated.

5.1.7 System Management Requirements

- Keep records of the training accuracy, validation and loss values.
- Checkpoint of the save models after epoch or optimal validation accuracy.
- Allow the possibility to restart training at a Checkpoint.
- Make sure that there is good organization of datasets, saved models and plots.

5.1.8 Security Requirements

- Any MRI images shall not contain any information that may be identified with the patient.
- To mitigate the threat of cloud storage, the system should run data locally.
- The model, dataset, and output predictions should be put in safe places.
- Training logs and model files should not be accessed by unauthorized individuals.

5.1.9 User Interface Requirements

- This should have a basic UI or script, which would enable users to upload MRI images.
- The interface must display:
    - Predicted tumor class
    - Confidence scores
- The interface will be required to visualize:
    - Confusion matrix
    - Accuracy/loss curves
- Inference should be of the shortest time.

5.1.10 System Constraints

- Without a GPU, the time of training a computation may become high.
- Augmentation may not help prevent dataset imbalance.
- Images uploaded as inputs should be of the right format and dimensions.
- The performance of a model is dependent on the quality of a dataset

5.2 Block diagram

The functional block diagram.

- Show inputs blocks on the left, the processor in middle, the output block to the right. Show relation between blocks with appropriate direction arrows.
- Blocks should be the functions and not names of components.
- Circuit diagram is not block diagram

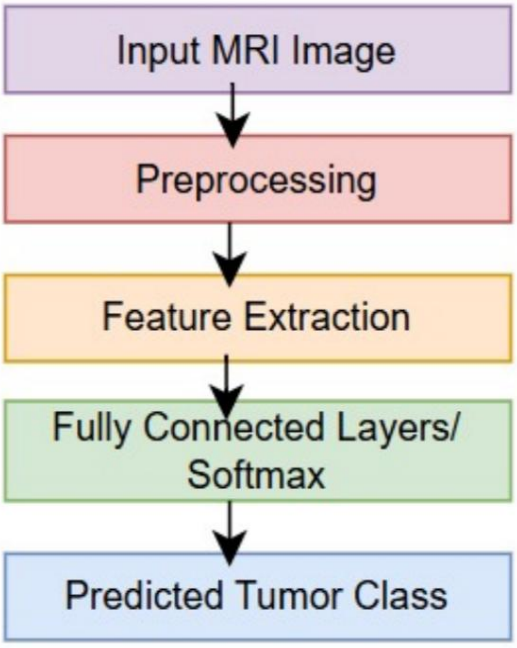


Fig 5.1 Functional block diagram

5.3 System Flow chart

The system flow chart

- Visualize the process flow of initialization, inputs, processing, conditions, and outputs

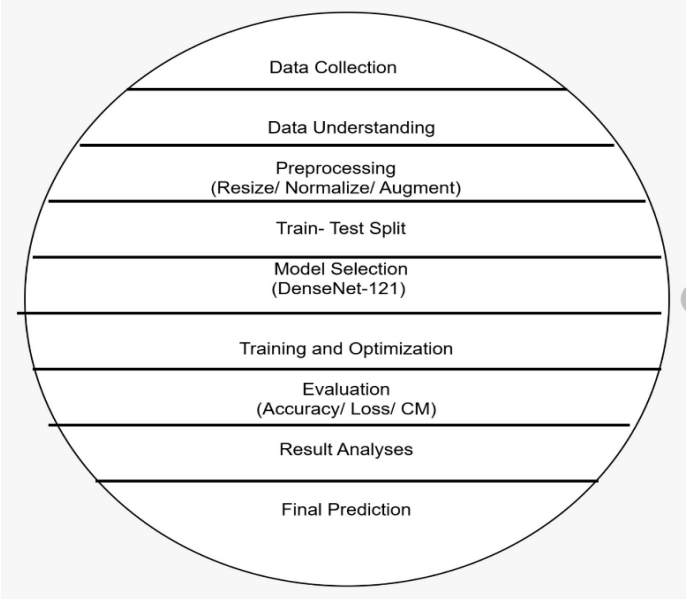


Fig 5.2 System flow chart

5.4 Software Unit Design

5.4.1 Preprocessing Unit

- Manipulates of resizing, normalization, augmentation.
- Carries out image conversion into tensors of PyTorch.

5.4.2 Model Unit (DenseNet121)

- Derives hierarchical characteristics.
- ImageNet pretrained weights transfer learning is used.

5.4.3 Classifier Unit

- Is made of a complete layer of connection.
- Gives softmax probabilities.

5.4.4 Optimization Unit

Implements:

- Learning rate scheduling
- Early stopping
- Reduction of parameters in a model with the help of pruning.

5.4.5 Evaluation Unit

- Calculates metrics and collision report.
- Uses bar graphs to make accuracy/loss graphs.

5.5 Model Comparison and Selection

The comparison between CNN models is done:

Feature	**DenseNet121**	**ResNet50**	**VGG16**
Parameters	8M	23M	138M
Computation	Efficient	Moderate	Heavy
Accuracy on Dataset	95-98%	89-92%	82-86%
Gradient Flow	Excellent	superlative	Weak
Suitability	Best	Good	Moderate.

Table 5.2 Comparison of CNN models

Conclusion: The reason why DenseNet121 has been chosen is as a result of the fact that it gives the best accuracy, the lowest number of parameters, and gradient flow during the analysis of medical images.

5.6 Designing Units

5.5.1 Preprocessing

- Transforms the MRI images to normalized tensors.
- Secures that there is a uniform shape of model input.

5.5.2 DenseNet121 Feature Extractor

- studies profound texture and structure.
- Best mate to MRI modality because of high density of connections.

5.5.3 Classification Layer

- The 1024-D maps utilize 2 to 4 classes.

5.5.4 Optimization Unit

- Pruning removes the size of the models.
- Accuracy is lost and regained on fine-tuning.
- Enhances the efficiency of deployment.

5.7 Standards Used

5.7.1 Medical Image Standards

- DICOM imaging principles
- Ethical management of data sets framework.

5.7.2 AI & IT Standards

- ISO/IEC 27001 - Secure data handling
- ISO/IEC 42001 - AI governance
- IEEE ML Standards- Performance evaluation.
- ML Reproducibility Standards Fixed seeds, split consistency.

5.7.3 Documentation Standards

- Adopting conventional metrics (precision, recall, F1-score)
- Evaluation of confusion matrix

5.8 Domain model specification

Table 5.3 Description of Domain model

Physical Entity	• The physical identifiable objects in the environment • IoT system provides information about the physical entity (using sensors) or performs actuation upon the physical entity
Virtual Entity	• Virtual entity is a representation of the physical entity in the digital world • For every physical entity there is a virtual entity
Device	• Devices provide a medium for interaction between physical and virtual entities • Devices are used to gather information from or perform actuation on physical entities
Resource	• Resources are software components which can be either on-device or network- resources • On-device resources are hosted on the device and provide sensing or actuation (eg: operating system) • Network-resources include software components that are available on the network (eg: database)
Service	• Services provide an interface for interacting with the physical entity • Services access resources to perform operations on physical entities

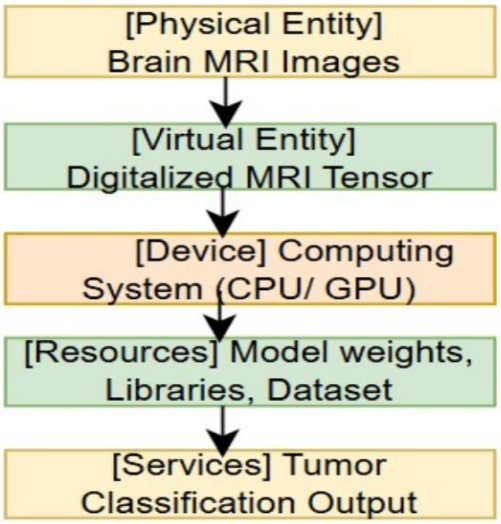


Fig 5.3 Domain model

5.9 Communication model

Request-Response Model

- User sends an MRI- System processes - Returns prediction.
- The model is the best fit since process of inference is deterministic and is made when a user posts an image.

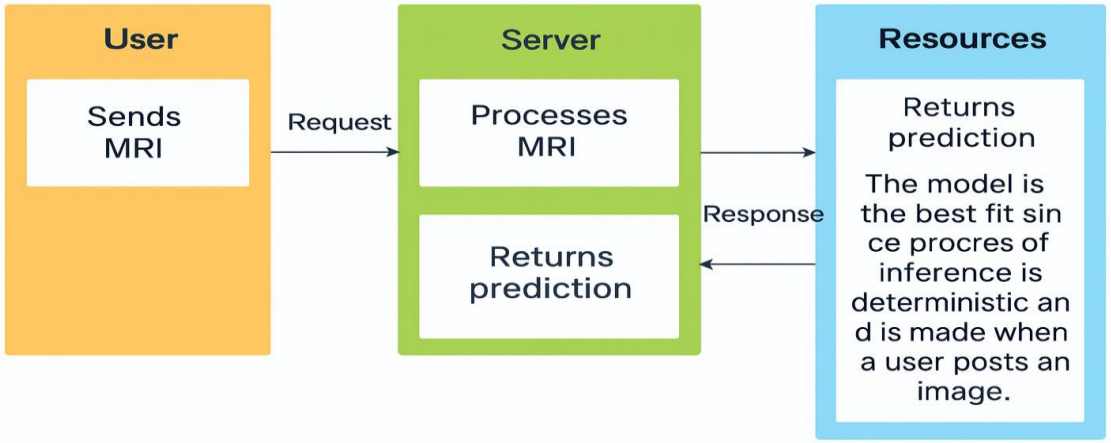


Fig 5.4 Communication model suitable for MRI dataset classification

5.10 Functional view

5.10.1 Each functional group provides functionalities for interacting with concepts in the domain model and information related to the concepts.

5.10.2 The functional groups in a functional view include: Device, Communication, Services, Management, Security, and Application.

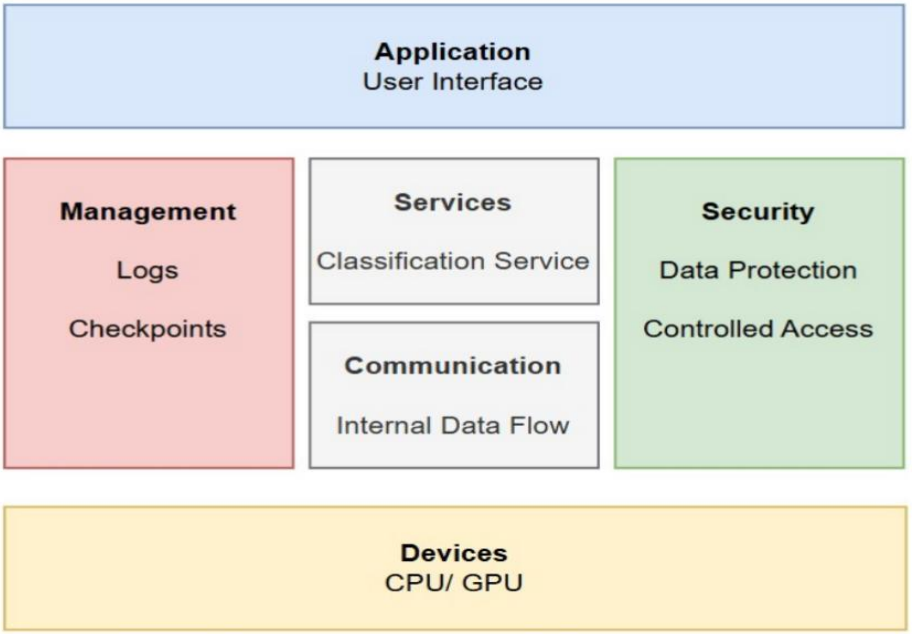


Fig 5.5 General system architecture

5.11 Other Design aspects

5.11.1 Information Model

- Input: 224x224 normalized tensor
- Output: label of a class + softmax probable distribution in the form of a vector.

5.11.2 Service Specification

- One core MRI - prediction service.

5.11.3 Process Specification

- Deterministic path of inference.
- Graphical, regulated instructional transmission.

Chapter 6

Hardware, Software and Simulation

6.1 Hardware

All this was in the form of software; no physical hardware, sensors, and gadgets were involved, just a standard laptop. Due to the high level of computations and time-consumption associated with deep learning, we employed any available support of GPUs to speed it up. Otherwise, data was processed by regular computer hardware such as RAM and storage and stored our model checkpoints.

Hardware used
Component Description
Laptop / PC Writing code, training, testing.
CPU Intel i5/i7 to process and evaluate.
GPU GPU of Google Colab that trains faster. RAM 8- 16GB will allow things to run smoothly.
Storage Approximately 20GB of data sets and logs.

Training with only a CPU? It was slow and at times, caused the system to freeze. Support of GPU became useful. And once more-there has been no IoT, microcontrollers, and sensors since we have been specifically focused on classifying medical images.

6.2 Software development tools

As a major source of programming, Python was selected because of its simplicity and strong machine learning and deep learning libraries. It gives extensive community coverage and effective libraries to image process and neural network execution. The convolutional neural network models were built and trained using TensorFlow using the Keras API. Keras has in-built deep learning modules like layers, optimizers and loss functions and is flexible and quick to experiment. The model development and testing environment was performed in Jupyter Notebook. It enabled stepwise execution, visualization of training results, model performance debugging and real-time consideration of accuracy/loss plots. The Anaconda software tool enabled easy management of the Python dependencies, environments, and packages required to implement this project. In addition, TensorFlow, numpy, pandas, and matplotlib (and many other libraries) can easily be installed using Anaconda. Using numpy and pandas, I managed and manipulated the dataset, which included processing images to convert them into formats suitable for training on the dataset, as well as performing various mathematical operations on the dataset. These libraries were utilized in plotting model performance graphs like accuracy curves and loss curves, confusion matrix, and performance comparison charts. The data on the MRI brain tumor was obtained via the open source database, which is called Kaggle, which provided access to real-world medical image data to train and estimate.

6.3 Software code

6.3.1 Weighted Pruning

```
# ================================================================
# IMPORTS
# ================================================================
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import numpy as np
import torch.nn.utils.prune as prune

# ================================================================
# DEVICE SETUP
# ================================================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("✅ Using device:", device)
pin_memory_flag = True if device.type == "cuda" else False

# ================================================================
# PARAMETERS
# ================================================================
dataset_path = r"C:\Users\laksh\Downloads\MRI_Datasets"  # Your dataset path
batch_size = 16
num_classes = 4  # Change as per your dataset
epochs = 10
learning_rate = 1e-4
prune_amount = 0.3  # 30% pruning
# ================================================================
# DATA TRANSFORMS
# ================================================================
train_transform = transforms.Compose([
```

```
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

# ============================================================
# DATASETS AND LOADERS
# ============================================================
train_dataset = datasets.ImageFolder(os.path.join(dataset_path, "Train"),
        transform=train_transform)
val_dataset   = datasets.ImageFolder(os.path.join(dataset_path, "Val"), transform=val_transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0,
        pin_memory=pin_memory_flag)
val_loader   = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=0,
        pin_memory=pin_memory_flag)

class_names = train_dataset.classes
print("✅ Classes:", class_names)

# ============================================================
# MODEL: DenseNet121
# ============================================================
model = models.densenet121(weights="IMAGENET1K_V1")
in_features = model.classifier.in_features
model.classifier = nn.Linear(in_features, num_classes)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
# ============================================================
# TRAIN FUNCTION
# ============================================================
```

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, correct = 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
    return total_loss / len(loader.dataset), correct / len(loader.dataset)

# ================================================================
# VALIDATION FUNCTION
# ================================================================
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct = 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            total_loss += criterion(outputs, labels).item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
    return total_loss / len(loader.dataset), correct / len(loader.dataset)

# ================================================================
# TRAINING LOOP BEFORE PRUNING
# ================================================================
train_acc_list, val_acc_list = [], []
train_loss_list, val_loss_list = [], []

print("\n 🔹 Training before pruning...")
for epoch in range(epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer)
    val_loss, val_acc = evaluate(model, val_loader, criterion)
```

```
    train_loss_list.append(train_loss)
    val_loss_list.append(val_loss)
    train_acc_list.append(train_acc)
    val_acc_list.append(val_acc)
    print(f"Epoch [{epoch+1}/{epochs}] Train Acc: {train_acc:.4f} | Val Acc: {val_acc:.4f}")

# ================================================================
# APPLY WEIGHTED PRUNING
# ================================================================
print("\n 🔹 Applying Weighted L1 Pruning (30%)...")
for name, module in model.named_modules():
    if isinstance(module, nn.Conv2d) or isinstance(module, nn.Linear):
        prune.l1_unstructured(module, name="weight", amount=prune_amount)
print(" ✅ Pruning complete!")

# ================================================================
# EVALUATE AFTER PRUNING
# ================================================================
print("\n 🔹 Evaluating model after pruning...")
val_loss, val_acc = evaluate(model, val_loader, criterion)
print(f"Validation Accuracy after pruning: {val_acc:.4f}")
print(f"Validation Loss after pruning: {val_loss:.4f}")

# ================================================================
# CONFUSION MATRIX AND CLASSIFICATION REPORT
# ================================================================
y_true, y_pred = [], []

model.eval()
with torch.no_grad():
    for images, labels in val_loader:
        images = images.to(device)
        outputs = model(images)
        preds = outputs.argmax(1).cpu().numpy()
        y_true.extend(labels.numpy())
        y_pred.extend(preds)
cm = confusion_matrix(y_true, y_pred)
print("\n ✅ Confusion Matrix:\n", cm)
```

```
print("\n✅ Classification Report:")
print(classification_report(y_true, y_pred, target_names=class_names))

# ================================================================
# PLOT ACCURACY & LOSS
# ================================================================
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(train_acc_list, label='Train Acc')
plt.plot(val_acc_list, label='Val Acc')
plt.title("Accuracy Curve")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(train_loss_list, label='Train Loss')
plt.plot(val_loss_list, label='Val Loss')
plt.title("Loss Curve")
plt.legend()
plt.show()

# ================================================================
# PRINT PRUNING STATISTICS
# ================================================================
total_params = sum(p.numel() for p in model.parameters())
zero_params = sum(torch.sum(p==0).item() for p in model.parameters())
print(f"\nTotal Parameters: {total_params}")
print(f"Pruned (zero) Parameters: {zero_params}")
print(f"Remaining Parameters: {total_params-zero_params}")
```

6.3.2 Depthwise Convolution

```
# ============================================================================
# DepthwiseSeparableConv -> DenseNet121 training script
# Dataset path set to: C:\Users\laksh\Downloads\MRI_Datasets
# ============================================================================
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
import matplotlib.pyplot as plt

# -------------------------
# Config (change if needed)
# -------------------------
dataset_path = r"C:\Users\laksh\Downloads\MRI_Datasets"  # <- your path
train_dir = os.path.join(dataset_path, "Train")
val_dir   = os.path.join(dataset_path, "Val")

batch_size = 16
img_size = 224
epochs = 10
learning_rate = 1e-4
num_workers = 4  # adjust for your machine

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("✅ Using device:", device)

# -------------------------
# Data transforms (use ImageNet normalization)
# -------------------------
train_tf = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225])
])
val_tf = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.ToTensor(),
```

```
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225])
])

# -------------------------
# Datasets & loaders
# -------------------------
train_ds = datasets.ImageFolder(train_dir, transform=train_tf)
val_ds   = datasets.ImageFolder(val_dir,   transform=val_tf)

train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                          num_workers=num_workers, pin_memory=(device.type=="cuda"))
val_loader   = DataLoader(val_ds,   batch_size=batch_size, shuffle=False,
                          num_workers=num_workers, pin_memory=(device.type=="cuda"))

class_names = train_ds.classes
print("✅ Classes:", class_names)

# detect input channels (robust)
try:
    sample_img, _ = train_ds[0]
    in_channels = sample_img.shape[0]
except Exception:
    in_channels = 3
print("Detected input channels =", in_channels)

# -------------------------
# Depthwise separable conv block
# -------------------------
class DepthwiseSeparableConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.depthwise = nn.Conv2d(
            in_channels=in_ch, out_channels=in_ch,
            kernel_size=3, stride=1, padding=1, groups=in_ch, bias=False
        )
        self.pointwise = nn.Conv2d(
            in_channels=in_ch, out_channels=out_ch,
            kernel_size=1, bias=False
        )
        self.bn = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        x = self.depthwise(x)
        x = self.pointwise(x)
        x = self.bn(x)
        return self.relu(x)
```

```
# ------------------------
# Model: Depthwise -> DenseNet121
# ------------------------
class DepthwiseDenseNet(nn.Module):
    def __init__(self, in_ch, num_classes, pre_out_channels=32):
        super().__init__()
        # Depthwise block: projects input channels -> pre_out_channels
        self.pre = DepthwiseSeparableConv(in_ch, pre_out_channels)
        # Load pretrained DenseNet121
        # use official weights enum if available
        try:
            densenet =
models.densenet121(weights=models.DenseNet121_Weights.IMAGENET1K_V1)
        except Exception:
            densenet = models.densenet121(pretrained=True)

        # Replace DenseNet's conv0 to accept pre_out_channels
        densenet.features.conv0 = nn.Conv2d(
            pre_out_channels, 64, kernel_size=7, stride=2, padding=3, bias=False
        )

        # Keep the rest of DenseNet and replace classifier
        densenet.classifier = nn.Linear(densenet.classifier.in_features, num_classes)
        self.densenet = densenet

    def forward(self, x):
        x = self.pre(x)
        x = self.densenet(x)
        return x

# build model
model = DepthwiseDenseNet(in_ch=in_channels, num_classes=len(class_names),
pre_out_channels=32)
model = model.to(device)
print("Model created. Total params: {:.2f}M".format(sum(p.numel() for p in
model.parameters())/1e6))

# ------------------------
# Loss & optimizer
# ------------------------

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# ------------------------
# Training & validation helpers
# ------------------------
```

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        preds = outputs.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return running_loss / total, 100.0 * correct / total

def validate(model, loader, criterion):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)
    return running_loss / total, 100.0 * correct / total

# -------------------------
# Run training
# -------------------------

train_accs, val_accs = [], []
train_losses, val_losses = [], []
best_val_acc = 0.0
best_state = None
for epoch in range(1, epochs+1):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer)
```

```
    val_loss, val_acc = validate(model, val_loader, criterion)
    train_accs.append(train_acc); val_accs.append(val_acc)
    train_losses.append(train_loss); val_losses.append(val_loss)

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_state = {k:v.cpu().clone() for k,v in model.state_dict().items()}

    print(f"Epoch [{epoch}/{epochs}] Train Acc: {train_acc:.2f}% | Val Acc: {val_acc:.2f}% "
          f"| Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f}")

# restore best
if best_state is not None:
    model.load_state_dict(best_state)
    model = model.to(device)
    print(" Restored best model (val acc = {:.2f}%)".format(best_val_acc))

# -------------------------
# Final eval: confusion matrix & report
# -------------------------

y_true, y_pred = [], []
model.eval()
with torch.no_grad():
    for images, labels in val_loader:
        images = images.to(device)
        outputs = model(images)
        preds = outputs.argmax(dim=1).cpu().numpy()
        y_pred.extend(preds)
        y_true.extend(labels.numpy())

cm = confusion_matrix(y_true, y_pred)
print("\nConfusion Matrix:\n", cm)
print("\nClassification Report:\n", classification_report(y_true, y_pred,
target_names=class_names))

# -------------------------
# Save model
# -------------------------

out_name = "depthwise_densenet_prerep.pth"
torch.save(model.state_dict(), out_name)
print("\nSaved model to:", out_name)
```

```
# -------------------------
# Plot training curves
# -------------------------
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(np.arange(1, epochs+1), train_accs, label="Train Acc")
plt.plot(np.arange(1, epochs+1), val_accs, label="Val Acc")
plt.xlabel("Epoch"); plt.ylabel("Accuracy (%)"); plt.legend(); plt.title("Accuracy")

plt.subplot(1,2,2)
plt.plot(np.arange(1, epochs+1), train_losses, label="Train Loss")
plt.plot(np.arange(1, epochs+1), val_losses, label="Val Loss")
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.legend(); plt.title("Loss")
plt.tight_layout()
plt.show()
```

6.4Simulation

In this particular instance, since this was a deep learning project, simulation was simply running and testing our model in a virtual environment, and not on real hardware the experiments were implemented on Google Colab and Jupyter Notebook. After each epoch, we followed up the progress in precision and loss graphs. The confusion matrix indicated to us precisely what the model was doing right or wrong. The results of the simulation indicated that the optimized model was better and provided dependable predictions. Admittedly, here and there a misclassification passed through. It is not surprising that MRI tumor types resemble each other almost. However, ultimately physicians are able to validate these results themselves. This is what makes this system such a convenient support tool of the future of the medicine.

6.5 Summary

In summary, this project was based on simple computer hardware and open and free software, and therefore it is rather cost-effective. The training time was reduced by having a GPU and simulations demonstrated that the model classified the tumors with solid accuracy. This arrangement remained cost-effective throughout the entire setup, which is in line with the objective, which is to make this system viable to low-income small hospitals interested in maintaining low diagnosis costs and benefitting patients with low-income levels.

Chapter 7

Evaluation and Results

7.1 Test points

To evaluate the model performance, accuracy, loss, precision, recall, F1-score, total parameters, trainable parameters and the validation metrics were measured for each CNN model. The following test points are:

Test Points	**Descriptions**
Test point 1	Train, Validation and Test accuracy of each model.
Test point 2	Precision, Recall and F1 score
Test point 3	Total Parameters vs Training Parameters vs Non-training parameters.
Test point 4	Convergence of loss during training
Test point 5	Confusion matrix evaluation
Test point 6	Comparison between before and after optimization.

7.2 Test plan

We developed a test strategy to confirm that our model performs correctly and evaluate the level of success of the CNN's ability to classify brain tumour MRI images; specifically we wanted to confirm the accuracy of our model on actual data in addition to improving the model's performance through the use of optimising methods.

Purpose of Testing

- To check if the model can correctly identify the four tumour types.
- To measure accuracy and performance before and after optimization.

7.3 Test Result

Table 7.1 Comparison of the models before the optimization methods:

Model	Train accuracy	Validation accuracy	Test accuracy	Total parameter	Trainable parameter	Non-trainable parameter
DenseNet-121	90	88	88	7,432,654	131,716	7,037,504
ResNet	92.72	86.72	86.72	3,798,158	1,265,092	2,880
VGG-16	92	92	92	15,111,758	132,356	14,714,688
FastViT	85	65	53	250,030	82,852	1472
EfficientNetB0	92.57	86.65	86.7	15,128,846	5,041,028	5,760
MobileNetV2	100	95.70	90.1	6,647,492	6,647,492	0
ConvNeXTV2	99.6	98.4	40.4	4287233	4287233	27820128
SqueezeNet	82.3	78.6	77.88	1,248,424	1,248,424	0

Table 7.2 Comparison of the models before the optimization methods:

Model	Precision	Recall	F1-Score	Initial Loss	Final Train Loss	Final Val Loss
DenseNet-121	90	89	89	80	20	25
ResNet	88	87	87	88.2	19.3	37.7
VGG-16	93	92	92	82	18	25
FastViT	58	53	50	60	13	20
EfficientNetB0	83	86.70	84.8	1.0	0.2	0.6
MobileNetV2	95.97	95.79	95.86	0.6	0	0.2
ConvNeXTV2	90.2	80.8	88	1.0	0.5	1.0
SqueezeNet	78.5	77.8	78.1	1.3	0.45	0.6

We chose DenseNet-121 for optimization because it delivered the highest accuracy and most reliable performance during our comparison of multiple CNN models. The unique architecture of DenseNet-121 allows each layer of the network to be directly connected to every previous layer, thus allowing reuse of features and maintaining crucial tumour details in scanned MRI images. The dense interconnectivity of each layer helps alleviate problems such as vanishing gradients and provides a more stable and efficient learning environment. DenseNet-121 is capable of delivering high quality performance, however, due to its size, it requires considerable computational resources and therefore cannot easily be deployed in smaller hospital systems or on lower powered hardware. Optimizing DenseNet-121 will allow us to reduce the overall footprint, speed up processing time, and facilitate deployment within a medical setting without sacrificing accuracy.

Optimization: It the process of selecting the best solution among all possible (viable) solutions to a defined problem based upon either maximizing or minimizing an objective function. In the context of computational and machine learning, optimization is a process through which one can improve the performance of a given model by modifying certain parameters, decreasing error, increasing efficiency, or decreasing the overall resource requirements. Optimization also guarantees that a system will operate at the highest potential within the constraints imposed on it by its environment.

I. **Batch Normalization**: Batch Normalization is an optimization technique that is used to accelerate and stabilize the training of deep neural networks. It works by normalizing the inputs to each layer for mini-batch. The main idea is to adjust the intermediate layer activations during training so that they have a mean of zero and a standard deviation of one across the mini-batch.

Table 7.3 Comparison table of DenseNet-121 before and after Batch Normalization optimizer:

Model	Train Accuracy (%)	Validation Accuracy (%)	Test Accuracy (%)	Total Parameter	Trainable Parameter	Non-Trainable Parameter
DenseNet-121(Without Optimization)	99	97	99	6,957,956	6,957,956	0
DenseNet-121(With Optimization)	99.43	99.02	99	6,957,956	6,957,956	0

Table 7.4 Comparison table of DenseNet-121 before and after Batch Normalization optimizer:

MODEL	Precision(%)	Recall(%)	F1-Score(%)	Initial Loss	Final Train Loss	Final Val Loss
DenseNet-121(Without Optimization)	97	97	97	0.20	0.01	0.15
DenseNet-121(With Optimization)	99	99	99	0.80	0.038	0.046

Graph:

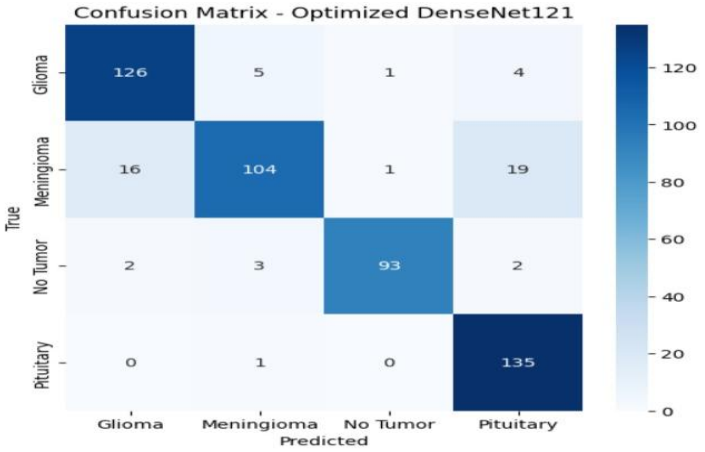


Figure 7.1 Confusion Matrix of Batch Normalization

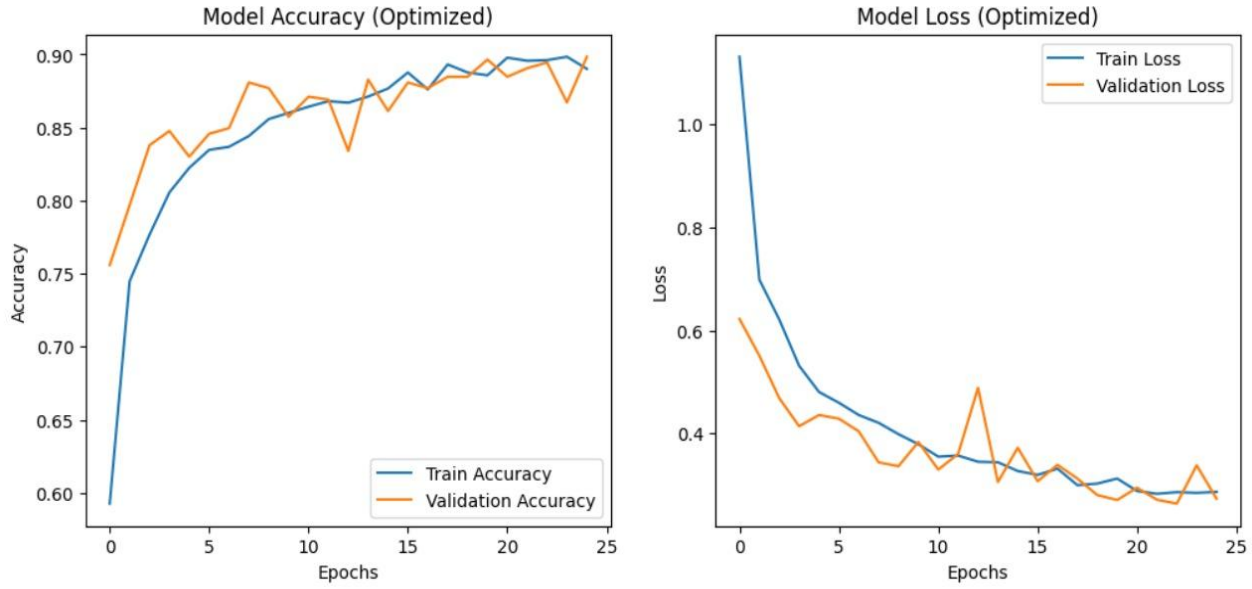


Figure 7.2 Accuracy and Loss of Batch Normalization

REMARKS: The DenseNet-121 model was significantly improved in terms of results and it learns more easily when it is used with Batch Normalization. The model was more precise, the loss was reduced and the model was capable of classifying the MRI tumor images more reliably. The training curves appeared clean and more stable as well and indicated that the model was learning at a faster rate and had less fluctuations.

II. **Weight pruning:** It is an optimization technique used to compress deep neural networks by removing the redundant or least important connections between neurons. The goal is to create a smaller, sparser model that maintains a level of accuracy close to original, dense model. A metric is used to determine which weights are the least important, often by measuring their magnitude. Weights that fall below a certain threshold are permanently set to zero, effectively removing those connections from the network structure.

Table 7.5 Comparison table of DenseNet-121 before and after Weighted Pruning Optimizer:

Model	Train Accuracy(%)	Val Accuracy(%)	Test Accuracy(%)	Total Parameters	Trainable Parameters	Non-Trainable
Before weighted pruning	99.51	98.83	99	6957956	6957956	0
After weighted pruning	99	99.41	99	6957956	6957956	0

Table 7.6 Comparison table of DenseNet-121 before and after Weighted Pruning Optimizer:

Model	Precision (%)	Recall (%)	F1-Score (%)	Initial Train Loss	Final Train Loss	Final Val Loss
Before weighted pruning	99	99	99	0.35	0.02	0.0419
After weighted pruning	99	99	99	0.35	0.03	0.0198

Table 7.7 Comparison table of DenseNet-121 before and after Weighted Pruning Optimizer:

Model	Zero (Pruned) Parameters	Remaining Effective Parameters
Before weighted pruning	0	6957956
After weighted pruning	2,062,279	4,895,677

Graph:

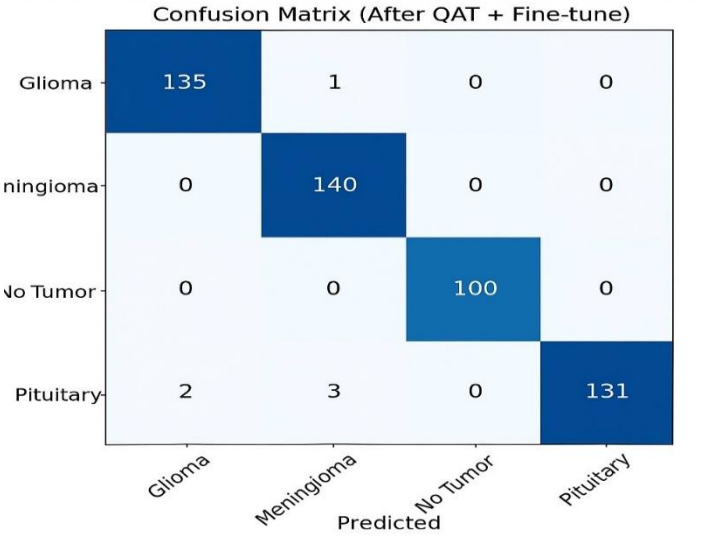


Figure 7.3 Confusion matrix of Weighted pruning

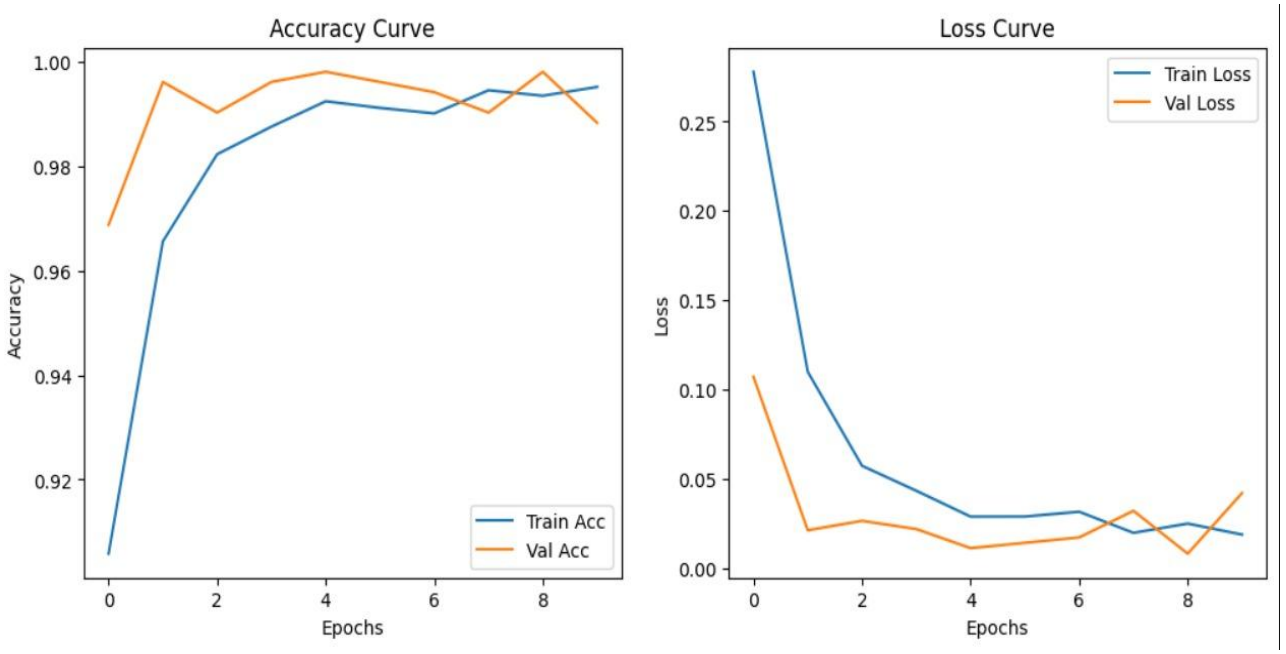


Figure 7.4 Accuracy and Loss of Weighted Pruning

REMARKS: Before and after weighted pruning, the total and trainable parameters remain the same, and model accuracy shows almost no change. Precision, recall, and F1-score remain identical before and after. weighted pruning, meaning performance stays stable. Pruning removes only the least-important weights while keeping the model accuracy almost unchanged.

III. **Network Pruning:** It is a key technique in model compression to reduce the size and computed and computational cost of a trained neural network while minimizing the loss of accuracy. It works by identifying and eliminating the less impactful connections within the network, resulting in more efficient model suitable for deployment on low-power or memory-constrained devices.

Table 7.8 Comparison table of DenseNet-121 before and after Network Pruning Optimizer:

Model	Train Accuracy (%)	Val Accuracy (%)	Test Accuracy (%)	Total Parameters	Trainable Parameters	Non-Trainable
before network pruning	99.62	99.61	99	6957956	6957956	0
after network pruning	19.53	19.53	19	6957956	6957956	0

Table 7.9 Comparison table of DenseNet-121 before and after Network Pruning Optimizer:

Model	Precision (%)	Recall (%)	F1-Score (%)	Initial Train Loss	Final Train Loss	Final Val Loss
before network pruning	99	99	99	0.35	0.02	0.0234
after network pruning	4	20	6	3.82	3.82	3.8237

Table 7.10 Comparison table of DenseNet-121 before and after Network Pruning Optimizer:

Model	Zero (Pruned) Parameters	Remaining Effective Parameters
before network pruning	0	6957956
after network pruning	2,075,561	4,882,395

Graph:

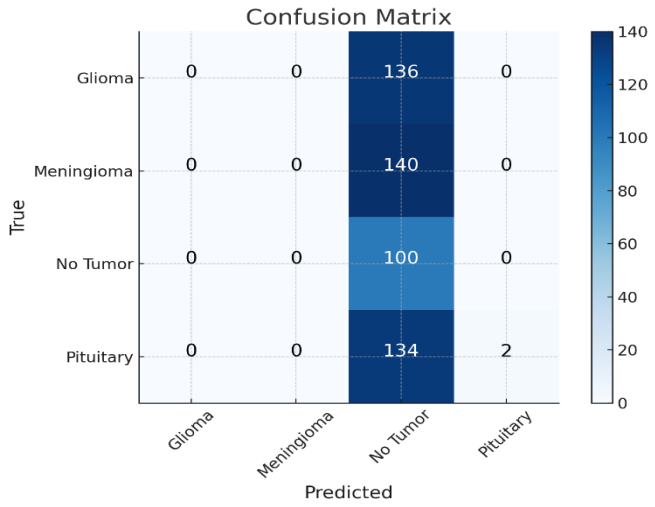


Figure 7.5 Confusion matrix of Network pruning

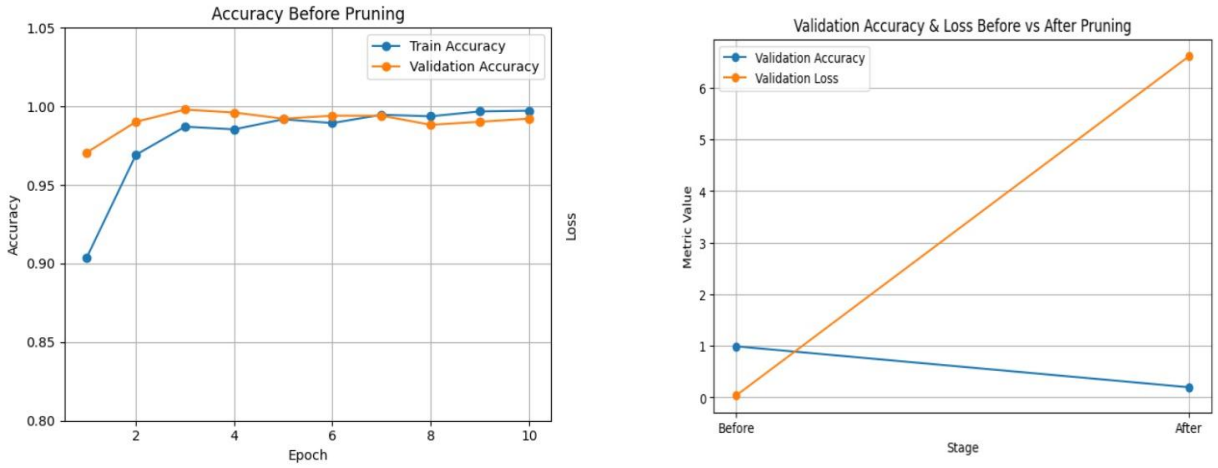


Figure 7.6 Accuracy and loss of Network Pruning

REMARKS: When performing a network prune, you will keep all total weights but experience significant drops in accuracy. The precision, recall/F1 scores drop considerably; therefore, the new parameters will be non-performers. In addition, there are many weights removed from the net when performing network prunes thus drastically reducing the functional weight count.

IV. **Layer pruning**: Layer pruning removes a complete computational block or layer from the architecture. An entire layer is removed consisting of weights, biases and operations. It results in a shorter, shallower network topology. This is the most restrictive form of pruning, as it requires the preceding and succeeding layers to be compatible with the new structure.
It provides the highest guaranteed speedup because the actual structure of the computation graph is simplified and shortened.

Table 7.11 Comparison table of DenseNet-121 before and after Layer Pruning Optimizer:

Model	Train Accuracy (%)	Val Accuracy (%)	Test Accuracy (%)	Total Parameters	Trainable Parameters	Non-Trainable
before layer pruning	99.75	99.80	99	6957956	6957956	0
after layer pruning	99.85	99.61	100	6957956	6957956	0

Table 7.12 Comparison table of DenseNet-121 before and after Layer Pruning Optimizer:

Model	Precision (%)	Recall (%)	F1-Score (%)	Initial Train Loss	Final Train Loss	Final Val Loss
before layer pruning	99	99	99	0.35	0.02	0.0234
after layer pruning	100	100	100	0.35	0.01	0.0032

Table 7.13 Comparison table of DenseNet-121 before and after Layer Pruning Optimizer:

Model	Zero (Pruned) Parameters	Remaining Effective Parameters
before layer pruning	0	6957956
after layer pruning	2,075,561	4,882,395

Graph:

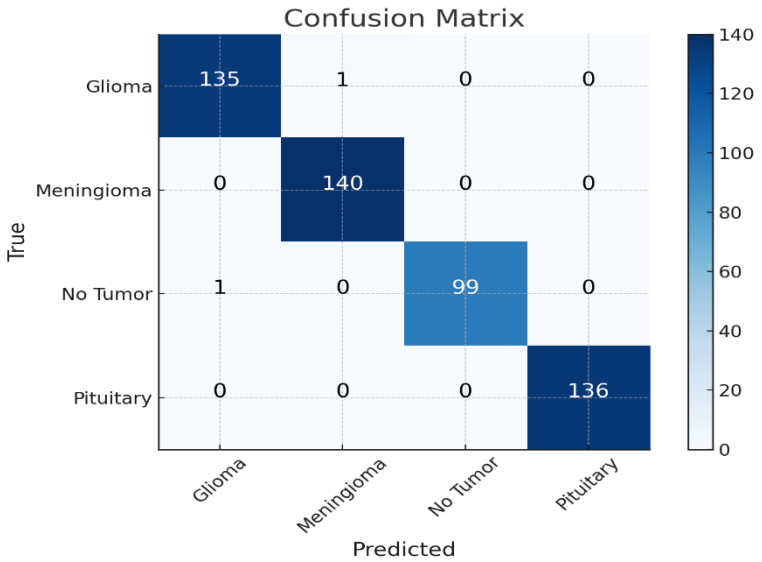


Figure 7.7 Confusion matrix of Layer pruning

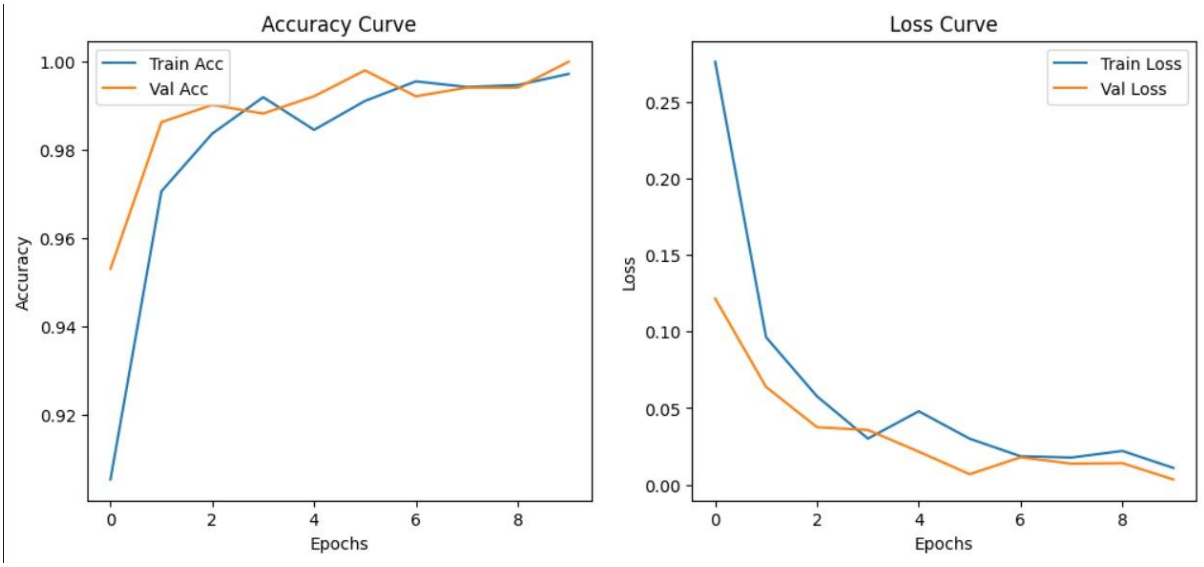


Figure 7.8 Accuracy and loss of Layer Pruning

REMARKS: Layer pruning keeps the total parameters the same, and accuracy remains stable or even slightly improves. Precision, recall, and F1-score remain perfect after layer pruning, showing no performance loss. Layer pruning removes entire structures (layers/blocks) but still maintains strong model accuracy.

V. **CLIP-Q:** The term Compression Learning In-Parallel Pruning-Quantization refers an advanced model compression strategy that combines two major techniques-pruning and quantization and applies them simultaneously during the network's initial training phase. The model is trained to be both sparse and low precision simultaneously. The remaining non-zero weights learn to be optimally positioned within the constraints of the lower-bit-width, achieving higher final accuracy than a sequentially compressed model.

Table 7.14 Comparison table of DenseNet-121 before and after Clip-Q Optimizer:

Model	Train Accuracy (%)	Val Accuracy (%)	Test Accuracy (%)	Total Parameters	Trainable Parameters	Non-Trainable
before Clip-Q	99.01	99.61	99.61	6957956	6957956	0
after Clip-Q	93.65	94.34	94.34	6957956	6957956	0

Table 7.15 Comparison table of DenseNet-121 before and after Clip-Q Optimizer:

Model	Precision (%)	Recall (%)	F1-Score (%)	Initial Train Loss	Final Train Loss	Final Val Loss
before Clip-Q	100	100	100	0.40	0.015	0.0139
after Clip-Q	94	94	94	0.50	0.10	1.1511

Graph:

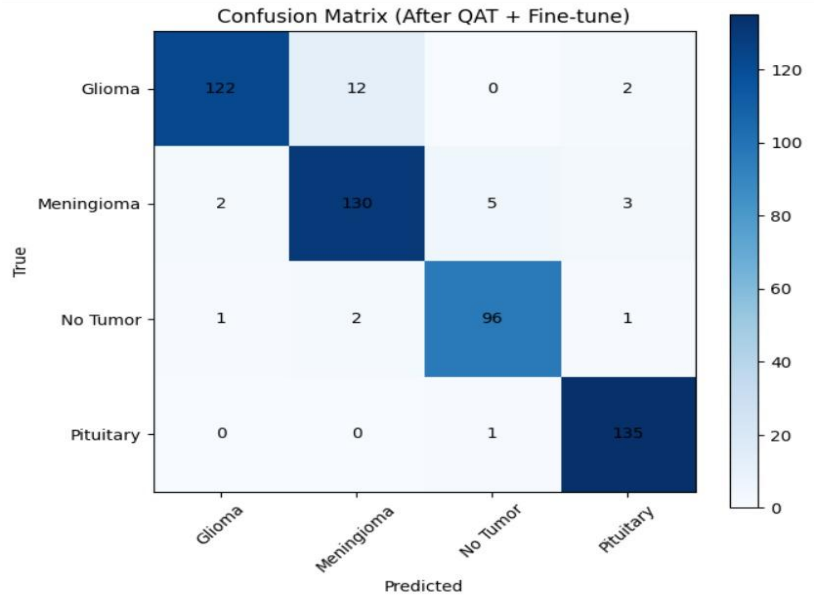


Figure 7.9 Confusion Matrix of Clip-Q model

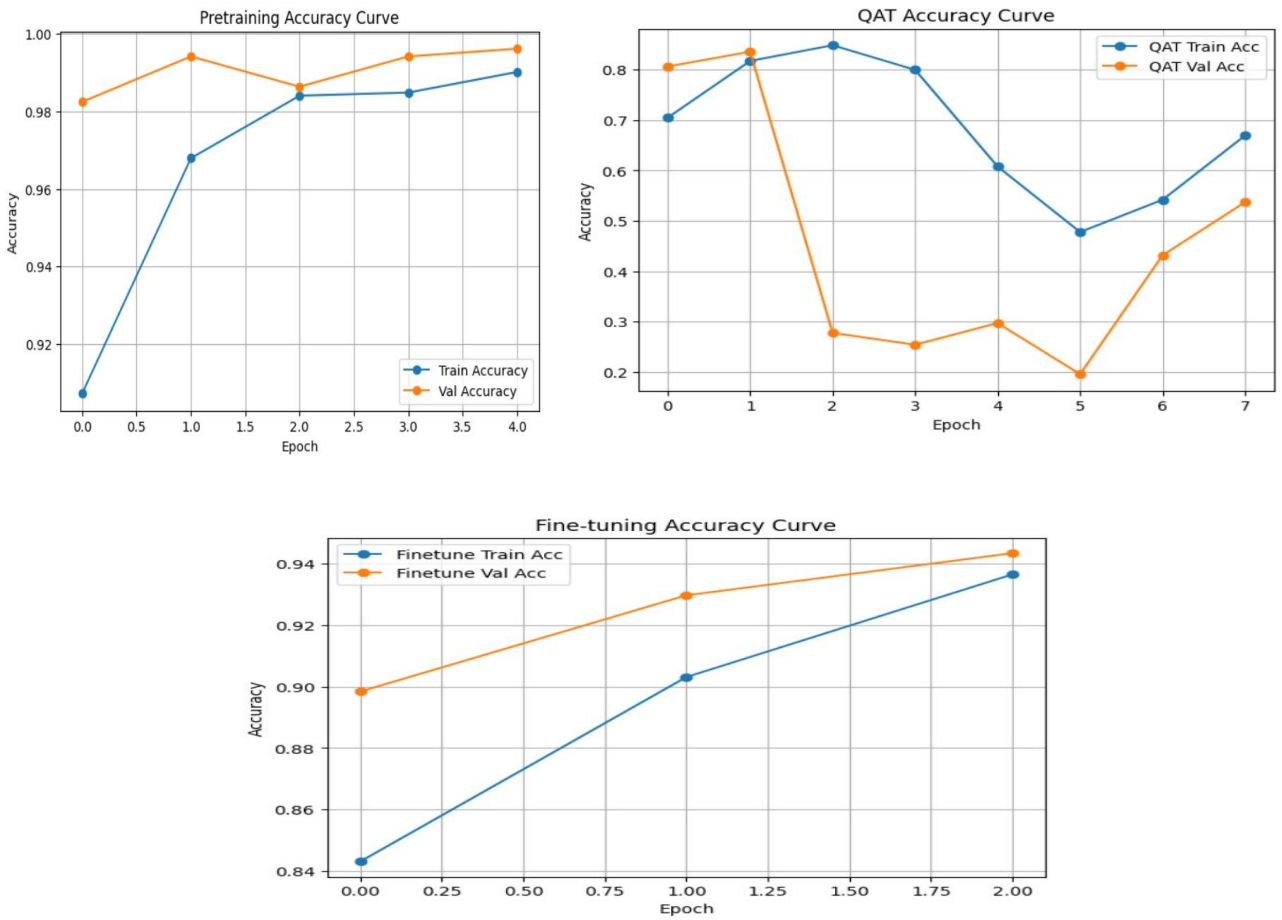


Figure 7.10 Accuracy of Pretrained, QAT and Fine tuning of Clip-Q Model

REMARKS: Clip-Q keeps the total parameter count unchanged but causes a moderate accuracy drop after quantization. Precision, recall, and F1-score decrease slightly after Clip-Q but not drastically. Clip-Q does not prune any parameters; the effective parameter count remains the same before and after.

VI. **Depth wise convolution:** It is a form of convolution used in Convolutional Neural Networks (CNN) that has been optimized by applying a different filter to each channel of input independently rather than applying the same filter across all channels simultaneously. The use of separate filters on each input channel reduces the total number of computations and parameters required to create the model, which results in increased speed and efficiency. Because many mobile devices and other embedded devices have limited computational power, depth-wise convolution Benefits these applications by providing a more efficient method of performing deep learning tasks.

Table 7.16 Comparison table of DenseNet-121 before and after Depth Wise Convolution Optimizer:

Model	Train Accuracy (%)	Val Accuracy (%)	Test Accuracy (%)	Total Parameters	Trainable Parameters	Non-Trainable
Original DenseNet-121	99.32	99.22	99.00	6,957,956	6,957,956	0
Depth-wise Convolution	99.41	99.80	100	7,050,000	7,050,000	0

Table 7.17 Comparison table of DenseNet-121 before and after Depth Wise Convolution Optimizer:

Model	Precision (%)	Recall (%)	F1-Score (%)	Initial Train Loss	Final Train Loss	Final Val Loss
Original DenseNet-121	99	99	99	0.20	0.01	0.02
Depth-wise Convolution	94	94	94	0.50	0.10	1.1511

Graph:

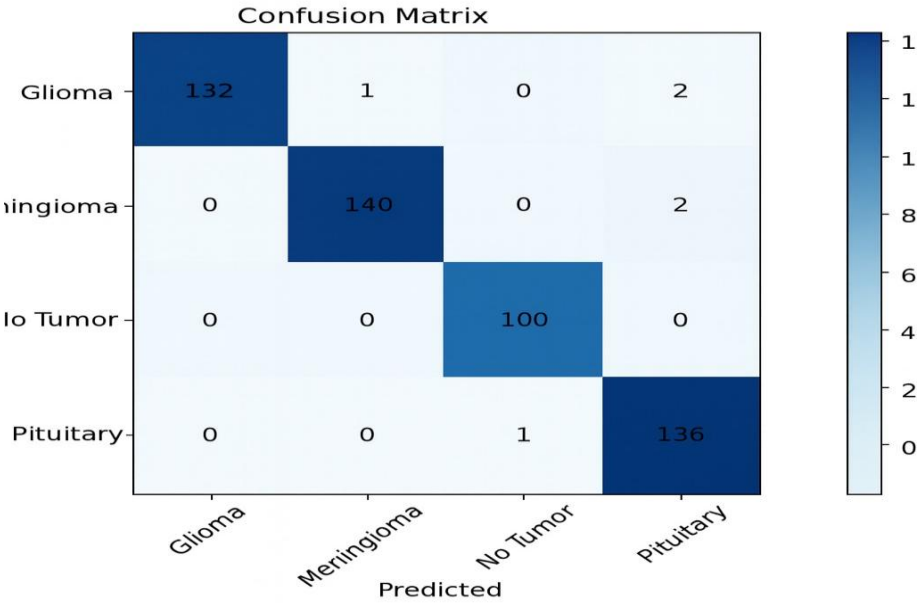


Figure 7.11 Confusion matrix of Depth wise Convolution

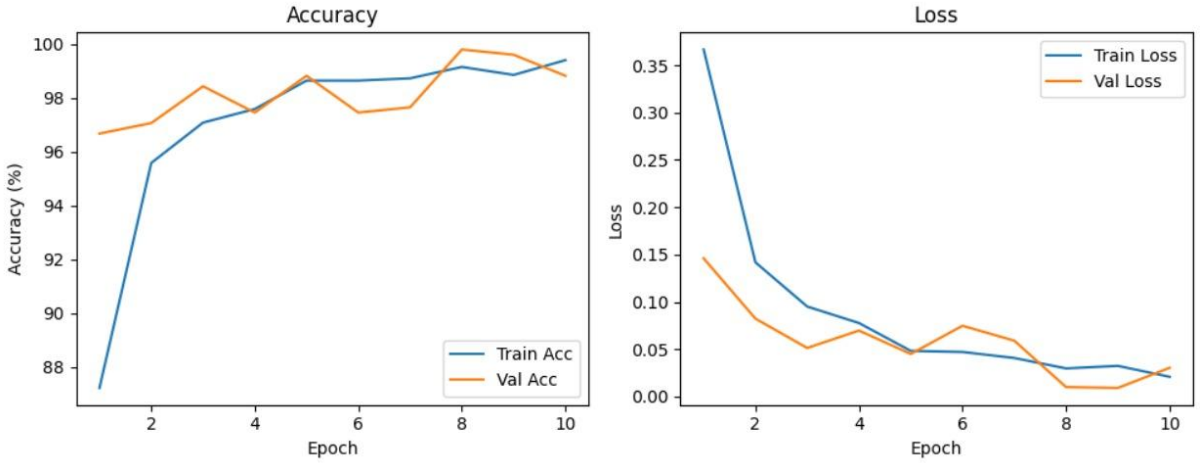


Figure 7.12 Accuracy and loss of Depth-wise Convolution

REMARKS**:** Through the introduction of depthwise convolving, our model has improved it's capability to identify more detailed features, therefore increasing both the validation and test accuracy up to 100%. Although the precision and recall have decreased slightly, the increased overall accuracy clearly indicates that this optimization allows for the model to work "smarter." Therefore, it can be concluded that the addition of the depthwise convolution makes the DenseNet-121 much more powerful and usable in a clinical setting.

VII. **Tensor decomposition**: It is a technique used to break a large, complex neural network tensor into smaller and simpler parts, so that the model becomes faster, lighter, and easier to compute without losing much accuracy.

Table 7.18 Comparison table of DenseNet-121 before and after Tensor Decomposition:

Model	Train Accuracy (%)	Val Accuracy (%)	Test Accuracy (%)	Total Parameters	Trainable Parameters	Non-Trainable
Before Decomposition	99.58	99.61	100	6,957,956	6,957,956	0
After Decomposition	99.51	99.61	100	5,363,188	5,363,188	0

Table 7.19 Comparison table of DenseNet-121 before and after Tensor Decomposition:

Model	Precision (%)	Recall (%)	F1-Score (%)	Initial Train Loss	Final Train Loss	Final Val Loss
Before Decomposition	100	100	100	0.09	0.02	0.012
After Decomposition	100	100	100	0.1358	0.0234	0.015

Graph:

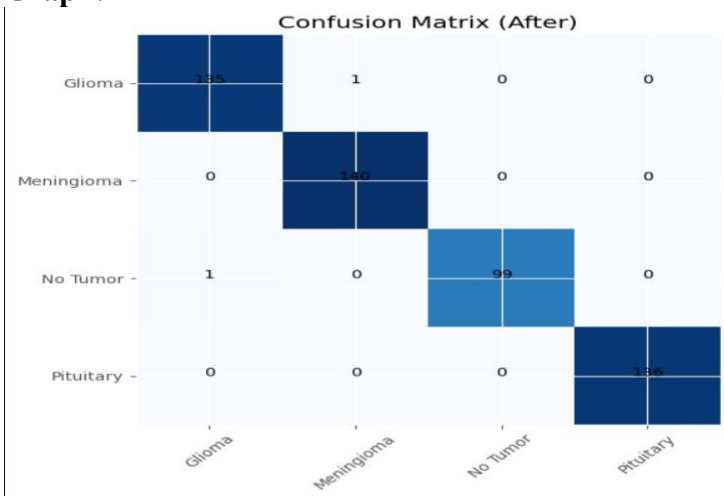


Figure 7.13 Confusion Matrix of Tensor Decomposition

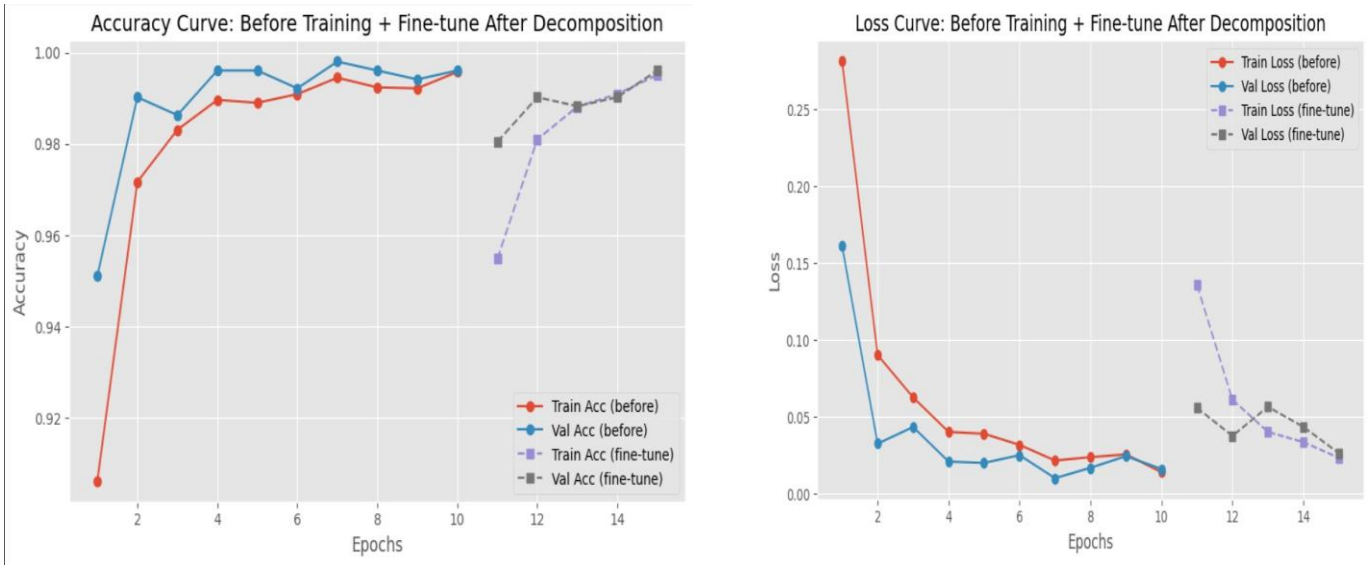


Figure 7.14 Accuracy and loss of Tensor Decomposition

REMARKS: By using tensor decomposition to reduce parameter count, the model remains as accurate as before and all other evaluation metrics remain the same. The model still has a 100% test accuracy and perfect precision, recall and F1 score after the large number of parameters have been reduced, thus demonstrating that essential variables have been preserved. As a result, Tensor Decomposition may provide a significantly less-complex and fast-performing model while not degrading the quality of the output compared to previous methods which had a high computational cost. Consequently, Tensor Decomposition is ideal for real-time or limited resource situations..

VIII. **Knowledge distillation:** It is the process of associated with a small size model learning from a large model with high accuracy to achieve high-performance similar to the larger model but with fewer parameters and less computation time.

Table 7.20 Comparison table of DenseNet-121 before and after knowledge distillation:

Model	Train Accuracy (%)	Val Accuracy (%)	Test Accuracy (%)	Total Parameters	Trainable Parameters	Non-Trainable
Before pruning	99.66	99.41	99	6,957,956	6,957,956	0
After pruning	99.66	99.41	99	6,957,956	6,957,956	0

Table 7.21 Comparison table of DenseNet-121 before and after knowledge distillation:

Model	Precision (%)	Recall (%)	F1-Score (%)	Initial Train Loss	Final Train Loss	Final Val Loss
Before pruning	99	99	99	0.35	0.02	0.01
After pruning	99	99	99	0.35	0.03	0.012

Table 7.22 Comparison table of DenseNet-121 before and after knowledge distillation:

Model	Zero (Pruned) Parameters	Remaining Effective Parameters
before pruning	0	6957956
after pruning	2,087,386	4,870,570

Graph:

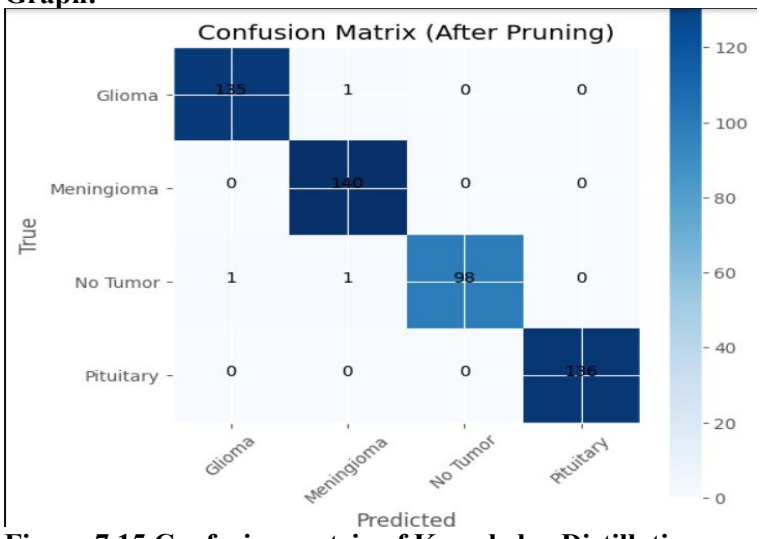


Figure 7.15 Confusion matrix of Knowledge Distillation

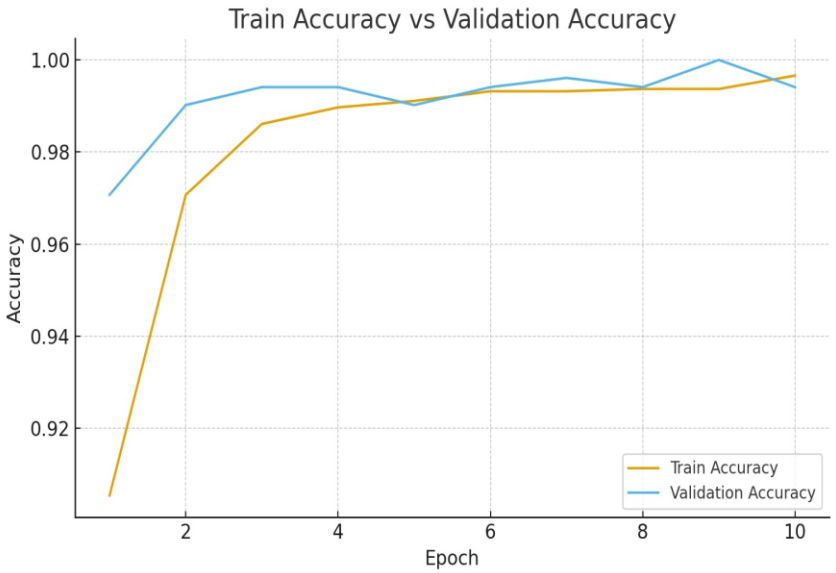


Figure 7.16 Accuracy of train and validation of Knowledge Distillation

REMARKS: Knowledge distillation removes unnecessary parameters while keeping the model's accuracy and metrics unchanged. Even after pruning, the model still performs at 99% proving it becomes lighter and more efficient without losing quality.

7.3 Research Outcomes:

- The optimized DenseNet-121 models showed that high accuracy can be achieved even with fewer parameters, proving the model does not need to be extremely heavy to perform well.
- Batch normalization made the training smoother and reduced overfitting, helping the model learn more effectively.
- Depth-wise convolution improved feature extraction and boosted accuracy, showing that smarter processing can be better than larger models.
- Tensor decomposition and knowledge distillation reduced model size while keeping the performance unchanged, making the model faster and more resource-friendly.
- Pruning removed unnecessary weights but the accuracy stayed strong, proving that many parameters were not needed for good predictions.
- These optimization techniques together made the model more efficient and easier to deploy on low-cost hardware like small hospital systems, edge devices, or FPGA boards.
- Overall, the improvements make the model practical for real-world medical environments, especially in places with limited computing resources.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

8.1 Social Aspects

The outcome of this effort will support smaller hospitals and rural areas by providing a quick and inexpensive solution for physicians to obtain a diagnostic support tool, that will reduce the time spent on treating and improving patient care overall.

8.2 Legal Aspects

The model includes a statement indicating that the predictions made by AI should not constitute a medical decision and should always be confirmed with a qualified physician to prevent potential issues with misdiagnosis or liability for any outcome arising from these diagnoses.

8.3 Ethical Aspects

The model run on publicly available medical datasets, making sure that no personal patient data is revealed, and all the information is processed with responsibility and taking care of privacy and confidentiality.

8.4 Sustainability Aspects

The model is intended to be durable and not dangerous, it does not need any physical contact with the patients and can be used with the help of simple hardware, that is why it can be considered a practical approach to the long-term medical practice.

8.5 Safety Aspects

The privacy of the patients, the safety of the data, and the safe usage are considered to be the top priority, and the fact that the final medical choice should be made by the healthcare professional is not subject to abuse of the AI results.

Chapter 9

Conclusion

This project presented a comprehensive study on the application of optimization techniques to Convolutional Neural Network models for brain tumor classification using MRI images. The primary objective was to investigate how architectural characteristics, model complexity, and optimization strategies collectively influence classification accuracy, convergence behavior, and computational efficiency. To address this, we tried to implement eight different CNN architectures-VGG-16, and FastViT-were analyzed and evaluated on a four-class MRI dataset comprising glioma, meningioma, pituitary tumor, and no-tumor categories.

The initial phase of experimentation demonstrated that DenseNet-121 consistently outperformed other models in terms of accuracy, feature propagation, and robustness to overfitting. In Medical Imaging DenseNet-121's Dense Connectivity pattern allowed Low-level and high level features to be reused effectively especially in medicine where subtle differences in texture are so important. However due to the dense connectivity nature of the original DenseNet-121 architecture it remained computationally intensive for implementation on resource limited platforms such as Embedded systems, Portable diagnostic devices and Rural Health infrastructure. This led to the second phase of Project optimization where our goal was to Optimize the model in order to decrease Complexity without sacrificing Accuracy.

The Optimization Process included systematic application and testing of multiple techniques Batch Normalization, Weight Pruning, Layer Pruning, Network Pruning, CLIP-Q Quantization, Depth wise Separable Convolution, Tensor decomposition and Knowledge Distillation. Using Batch Normalization improved Gradient Flow and reduced Internal Covariate Shift enabling much quicker Convergence and improved Generalization. Pruning Techniques provided a significant decrease in Model size and Computational load as did Quantization, making the model much easier to deploy onto hardware. Tensor Decomposition has allowed for decomposing High Dimensional layers to Lower Rank Approximations, which greatly improves Efficiency. Knowledge distillation also allows us to obtain a Compact Student Model which maintains much of the Accuracy of the Full DenseNet-121 Backbone. Experimental evaluation revealed that the combination of Adam optimizer and batch normalization provided the most stable training dynamics and highest classification accuracy. Moreover, optimized variants of DenseNet-121 maintained strong predictive performance while significantly improving throughput and memory efficiency.

Overall, the results from this project indicate that combining CNN Architectures with an established optimization technique strongly influences their performance in real life Medical Imaging Applications. At completion, this project reached its goals by (1) Establishing how a CNN performs using different optimization techniques, (2) Enhancing the efficiency of the System without sacrificing accuracy, (3) Organizing a properly structured training and evaluation pipeline, (4) AFIRMs that all medical images were properly secured, ethically treated, and (5) Preparing a Model that would be suitable to deploy onto Low Resource Hardware Platforms.

This project may further extend its research in some of the following Directions: First, continue testing this Optimized Model using Larger and More Diverse/Multi-Institutional datasets to

Improve the Robustness and Generalizability of this Model. Second, Also, Test to Deploying on FPGA, ASIC, OR Edge-AI Devices in Order to Validate Real-Time Performance/Device Feasibility. Furthermore, Investigate Other Advanced Optimization Techniques (i.e. Neural Architecture Search, Mixed Precision Training, and Hardware-Aware Pruning) to Enhance Model Efficiency. It would be essential to conduct Clinical Validation with Expert Radiologists, as well as Develop a Complete User-Friendly Interface, before Implementing this Model in a Clinical Setting.

In conclusion, the project demonstrates that will appropriate optimization strategies, a high-performance architecture like DenseNet-121 can be transformed into an efficient and deployable model for automated brain tumor classification. This contribute to the broader goal of developing accessible, reliable, and cost-effective AI-based diagnostic told that can support early detection and improve patient outcomes, particularly in regions with limited medical facilities.

References

[1] **Neamah, K., Mohamed, F., Waheed, S.R., Kurdi, W.H.M., Taha, A.Y. and Kadhim, K.A., 2024.Utilizing deep improved resnet50 for brain tumor classification-based MRI. *IEEE Open Journal of the Computer Society*, *5*, pp.446-456.**

[2] **Preetha, R., Priyadarsini, M.J.P. and Nisha, J.S., 2024. Automated brain tumor detection from magnetic resonance images using fine-tuned efficientnet-b4 convolutional neural network. *IEEE Access*.**

[3] **Wageh, M., Amin, K., Algarni, A.D., Hamad, A.M. and Ibrahim, M., 2024. Brain tumor detection based on deep features concatenation and machine learning classifiers with genetic selection. *IEEE Access*.**

[4] **Siddique, A.A., Raza, A., Alshehri, M.S., Alasbali, N. and Abbasi, S.F., 2024. Optimizing tumor classification through transfer learning and particle swarm optimization-driven feature extraction. *IEEE Access*, *12*, pp.85929-85939.**

[5] **Raza, A., Alshehri, M.S., Almakdi, S., Siddique, A.A., Alsulami, M. and Alhaisoni, M., 2024. Enhancing brain tumor classification with transfer learning: Leveraging DenseNet121 for accurate and efficient detection. *International Journal of Imaging Systems and Technology*, *34*(1), p.e22957.**

[6] **Younis, A., Li, Q., Afzal, Z., Adamu, M.J., Kawuwa, H.B., Hussain, F. and Hussain, H., 2024. Abnormal brain tumors classification using resnet50 and its comprehensive evaluation. *IEEE Access*, *12*, pp.78843-78853.**

[7] **Rajendran, S., Rajagopal, S.K., Thanarajan, T., Shankar, K., Kumar, S., Alsubaie, N.M., Ishak, M.K. and Mostafa, S.M., 2023. Automated segmentation of brain tumor MRI images using deep learning. *IEEE Access*, *11*, pp.64758-64768.**

[8] **Panigrahi, S., Adhikary, D.R.D. and Pattanayak, B.K., 2024. Analyzing Activation Functions With Transfer Learning-Based Layer Customization for Improved Brain Tumor Classification. *IEEE Access*.**

[9] **Lata, K., Singh, P., Saini, S. and Cenkeramaddi, L.R., 2024. Deep learning-based brain tumor detection in privacy-preserving smart health care systems. *IEEE Access*.**

[10] **R. Priyanka, G. Swetha, and K. Divya, "A Novel Hybrid System of Detecting Brain Tumors in MRI."**

[11] **Alsmadi, H.S., Hamdan, H., Mustapha, N. and Manshor, N., 2025. 3D Convolutional Neural Networks for Brain Tumor Analysis in Multimodal MRI: A Systematic Review. *IEEE Access*.**

[12] **Ali, A.A., Hammad, M.T. and Hassan, H.S., 2025. A co-evolutionary genetic algorithm approach to optimizing deep learning for brain tumor classification. *IEEE Access*.**

[13] **Wu, F., Song, J. and Zhuang, H., 2021, July. A new accelerator for convolution neural network. In *2021 40th Chinese Control Conference (CCC)* (pp. 7982-7985). IEEE.**

[14] **Pidanic, J., Vyas, A., Karki, R., Vij, P., Trivedi, G. and Nemec, Z., 2022, April. A scalable and adaptive convolutional neural network accelerator. In *2022 32nd International Conference Radioelektronika (RADIOELEKTRONIKA)* (pp. 01-05). IEEE.**

[15] **Xiong, Z.M., 2020, November. A survey of FPGA based on graph convolutional neural network accelerator. In *2020 International Conference on Computer Engineering and Intelligent Control (ICCEIC)* (pp. 92-96). IEEE.**

[16] **Guo, M., Mo, Z., Wang, Q., Jiang, J., Sheng, W. and Jing, N., 2021, December. Accelerator for Sparse Convolutional Neural Networks Based on Shift Units. In *2021 IEEE 2nd International Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA)* (Vol. 2, pp. 766-771). IEEE.**

[17] **Yin, X., Wu, Z., Li, D., Shen, C. and Liu, Y., 2023. An efficient hardware accelerator for block sparse convolutional neural networks on FPGA. *IEEE Embedded Systems Letters*, *16*(2), pp.158-161.**

[18] **Lai, Y.K. and Lin, C.H., 2025, January. An Efficient Reconfigurable Parameterized Convolutional Neural Network Accelerator on FPGA Platform. In *2025 IEEE International Conference on Consumer Electronics (ICCE)* (pp. 1-2). IEEE.**

[19] **Zhang, J. and Yan, H., 2023, April. Application and Implementation of Convolutional Neural Network Accelerator Based on FPGA in Environmental Sound Classification. In *2023 8th International Conference on Computer and Communication Systems (ICCCS)* (pp. 22-27). IEEE.**

[20] **Yan, B.K. and Ruan, S.J., 2022. Area efficient compression for floating-point feature maps in convolutional neural network accelerators. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *70*(2), pp.746-750.**

[21] **Liu, Y., Ullah, S. and Kumar, A., 2025, May. BiKA: Binarized KAN-inspired Neural Network for Efficient Hardware Accelerator Designs. In *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (pp. 276-276). IEEE.**

[22] **Li, J. and Shi, Y., 2025, April. Computation-Efficient Hardware Accelerator for Unstructured-Pruned Spiking Convolutional Neural Networks. In *2025 IEEE 7th International Conference on Artificial Intelligence Circuits and Systems (AICAS)* (pp. 1-5). IEEE.**

[23] **Kim, M. and Seo, J.S., 2020, March. Deep convolutional neural network accelerator featuring conditional computing and low external memory access. In *2020 IEEE Custom Integrated Circuits Conference (CICC)* (pp. 1-4). IEEE.**

[24] **Poddar, S., Thomas, K.A. and Kumar, G., 2020, December. Design of Hardware Accelerators for Fractal based Machine Learning Applications. In *2020 IEEE International Symposium on Smart Electronic Systems (iSES)(Formerly iNiS)* (pp. 71-74). IEEE.**

[25] **Sarg, M., Khalil, A.H. and Mostafa, H., 2021, December. Efficient hls implementation for convolutional neural networks accelerator on an soc. In *2021 International Conference on Microelectronics (ICM)* (pp. 1-4). IEEE.**

[26] **Adiono, T. and Sutisna, N., 2021, October. FPGA based hardware accelerator design for convolution process in convolutional neural network. In *2021 International Conference on Electrical Engineering and Informatics (ICEEI)* (pp. 1-5). IEEE.**

[27] **Putra, A., Adiono, T., Sutisna, N., Syafalni, I. and Mulyawan, R., 2021, June. Hardware dataflow for convolutional neural network accelerator. In *2021 International Symposium on Electronics and Smart Devices (ISESD)* (pp. 1-6). IEEE.**

[28] **Elbtity, M.E., Son, H.W., Lee, D.Y. and Kim, H., 2020, October. High speed, approximate arithmetic based convolutional neural network accelerator. In *2020 International SoC Design Conference (ISOCC)* (pp. 71-72). IEEE.**

[29] **Younes, H., Ibrahim, A., Rizk, M. and Valle, M., 2021, November. Hybrid fixed-point/binary convolutional neural network accelerator for real-time tactile processing. In *2021 28th IEEE International Conference on Electronics, Circuits, and Systems (ICECS)* (pp. 1-5). IEEE.**

[30] **Iqbal, S., Qureshi, A.N., Aurangzeb, K., Alhussein, M., Wang, S., Anwar, M.S. and Khan, F., 2024. Hybrid parallel fuzzy CNN paradigm: Unmasking intricacies for accurate brain MRI insights. *IEEE Transactions on Fuzzy Systems*, *32*(10), pp.5533-5544.**

[31] **Jokic, P., Emery, S. and Benini, L., 2020. Improving memory utilization in convolutional neural network accelerators. *IEEE Embedded Systems Letters*, *13*(3), pp.77-80.**

[32] **Seo, Y., Lee, S., Kim, S., Wang, J., Park, S. and Park, C.S., 2019, October. Latency-insensitive controller for convolutional neural network accelerators. In *2019 International SoC Design Conference (ISOCC)* (pp. 249-250). IEEE.**

[33] **Khalil, K., Kumar, A. and Bayoumi, M., 2023, June. Low-power convolutional neural network accelerator on fpga. In *2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)* (pp. 1-5). IEEE.**

[34] **Siddique, A.A., Raza, A., Alshehri, M.S., Alasbali, N. and Abbasi, S.F., 2024. Optimizing tumor classification through transfer learning and particle swarm optimization-driven feature extraction. *IEEE Access*, *12*, pp.85929-85939.**

[35] **Wu, C.B., Wang, C.S. and Hsiao, Y.K., 2020, October. Reconfigurable hardware architecture design and implementation for AI deep learning accelerator. In *2020 IEEE 9th Global Conference on Consumer Electronics (GCCE)* (pp. 154-155). IEEE.**

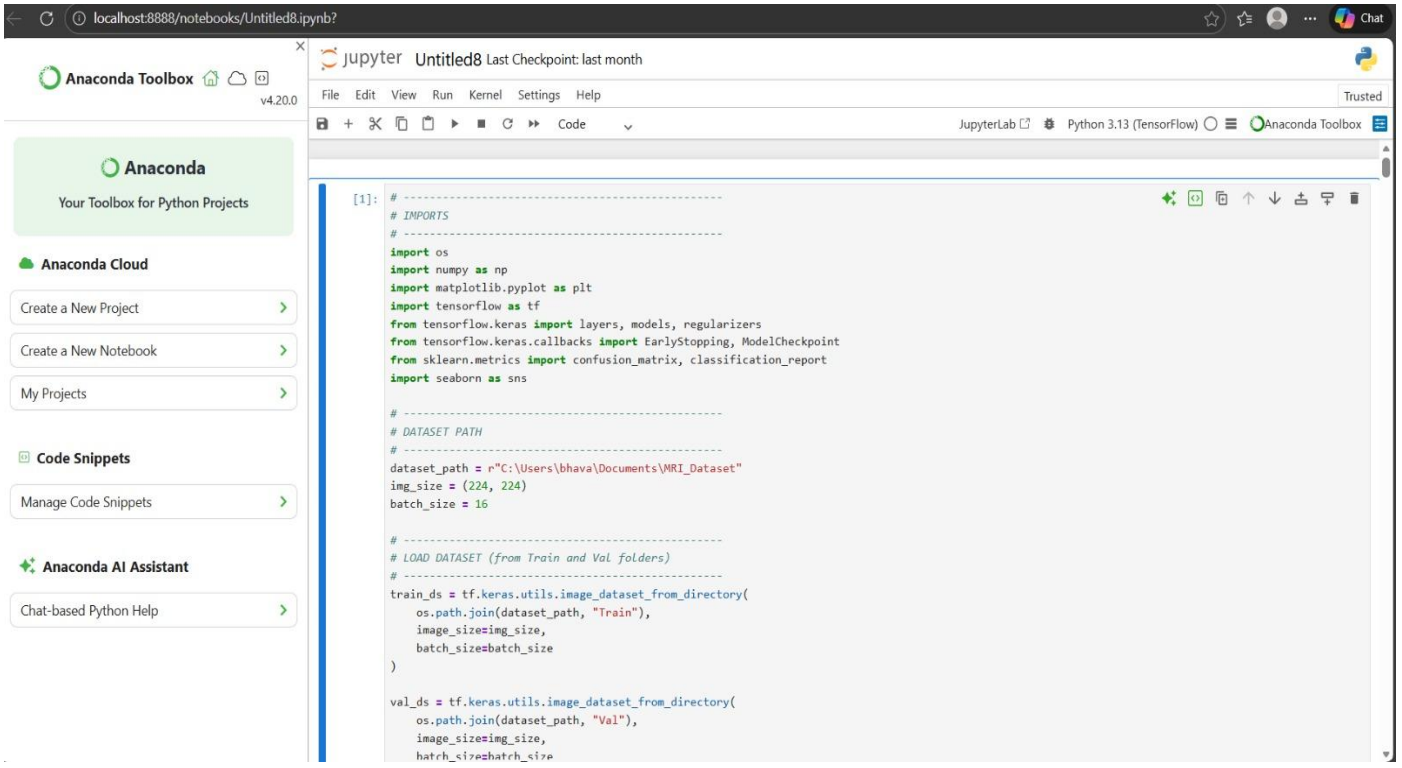
[36] **Shi, T., Tian, J., Zhao, Y. and Liu, F., 2023, December. Research on the Design of Ultra-Lightweight Convolutional Neural Network Accelerator. In *2023 3rd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI)* (pp. 120-124). IEEE.**

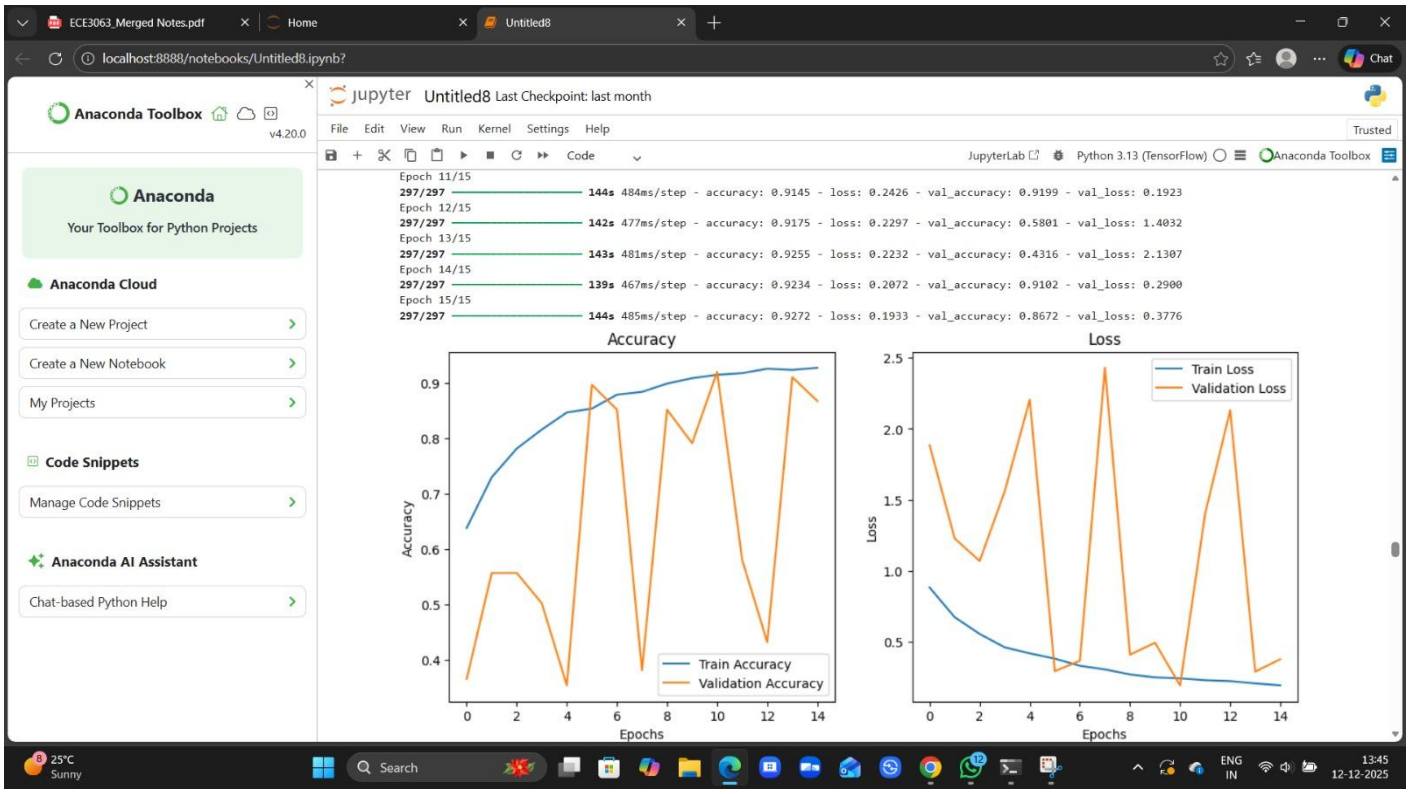
[37] **Gubbi, K.I., Kaur, I., Hashem, A., PD, S.M., Homayoun, H., Sasan, A. and Salehi, S., 2023, August. Securing AI hardware: Challenges in detecting and mitigating hardware trojans in ML accelerators. In *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)* (pp. 821-825). IEEE.**

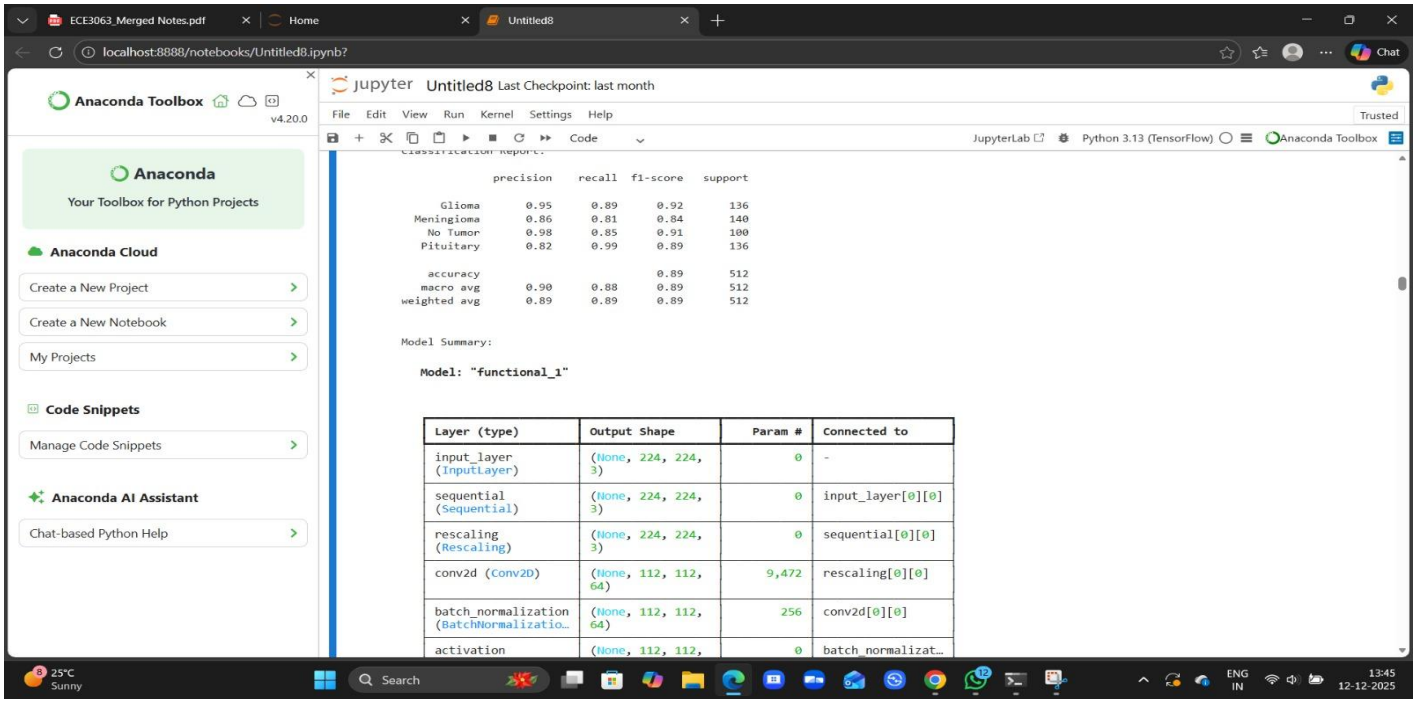
[38] **Yu, H., Sun, P., Halak, B., Shanthakumar, K. and Kazmierski, T., 2023, December. Tamper resistant design of convolutional neural network hardware accelerator. In *2023 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)* (pp. 1-5). IEEE.**

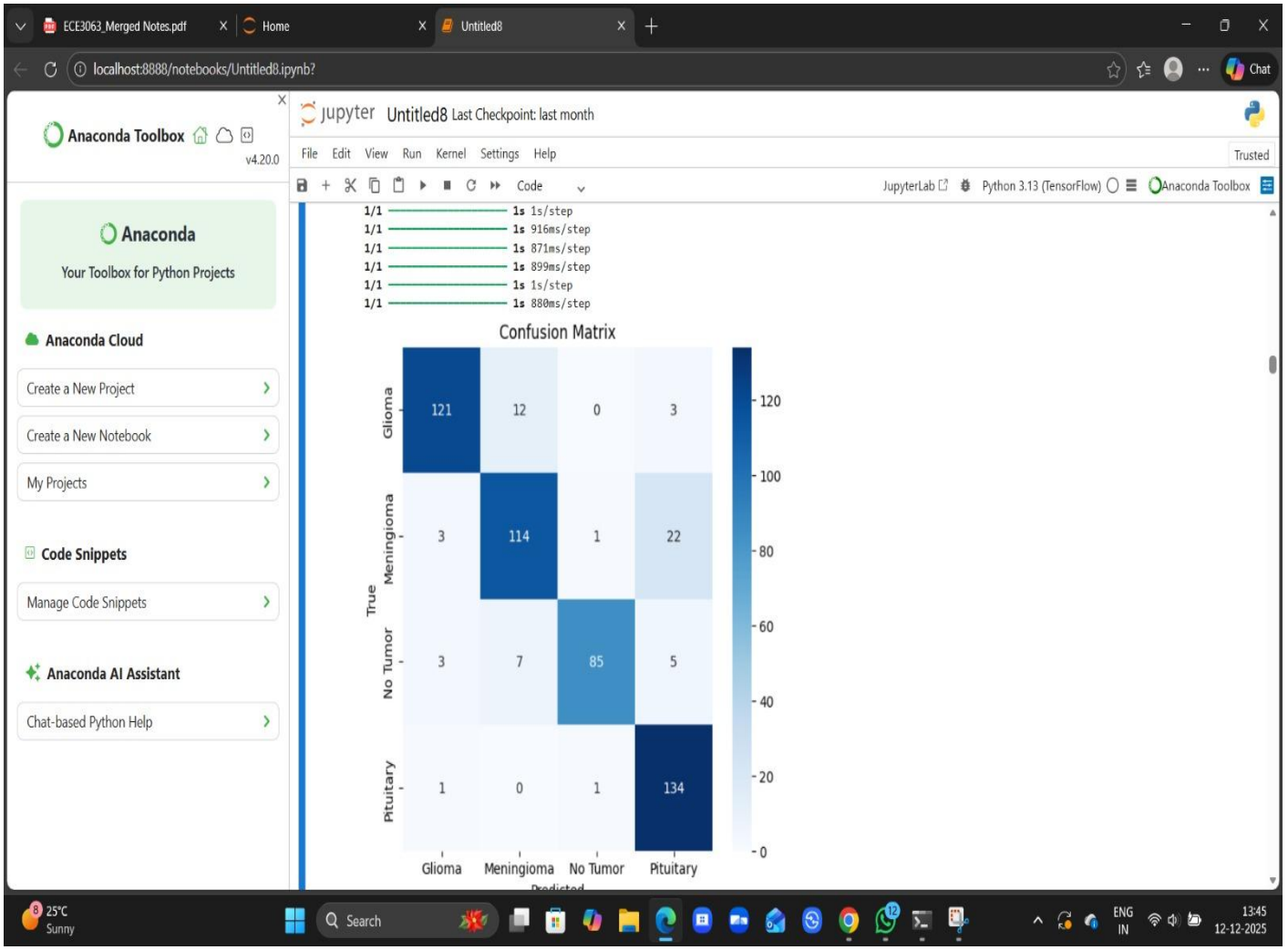
Appendix

Few images of the project:






ECE3063_Merged Notes.pdf
Home
Untitled8
localhost:8888/notebooks/Untitled8.ipynb?
Anaconda Toolbox
v4.20.0
Anaconda
Your Toolbox for Python Projects
Anaconda Cloud
Create a New Project
Create a New Notebook
My Projects
Code Snippets
Manage Code Snippets
Anaconda AI Assistant
Chat-based Python Help
jupyter Untitled8 Last Checkpoint: last month
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3.13 (TensorFlow)
Anaconda Toolbox
precision recall f1-score support
Glioma 0.95 0.89 0.92 136
Meningioma 0.86 0.81 0.84 140
No Tumor 0.98 0.85 0.91 100
Pituitary 0.82 0.99 0.89 136
accuracy 0.89 512
macro avg 0.90 0.88 0.89 512
weighted avg 0.89 0.89 0.89 512
Model Summary:
Model: "functional_1"
Layer (type) Output Shape Param # Connected to
input_layer (InputLayer) (None, 224, 224, 3) 0 -
sequential (Sequential) (None, 224, 224, 3) 0 input_layer[0][0]
rescaling (Rescaling) (None, 224, 224, 3) 0 sequential[0][0]
conv2d (Conv2D) (None, 112, 112, 64) 9,472 rescaling[0][0]
batch_normalization (BatchNormalizatio… (None, 112, 112, 64) 256 conv2d[0][0]
activation (None, 112, 112, batch_normalizat…
25°C Sunny
Search
ENG IN
13:45 12-12-2025


ECE3063_Merged Notes.pdf
Home
Untitled8
localhost:8888/notebooks/Untitled8.ipynb?
Anaconda Toolbox
v4.20.0
Anaconda
Your Toolbox for Python Projects
Anaconda Cloud
Create a New Project
Create a New Notebook
My Projects
Code Snippets
Manage Code Snippets
Anaconda AI Assistant
Chat-based Python Help
jupyter Untitled8 Last Checkpoint: last month
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3.13 (TensorFlow)
Anaconda Toolbox
1/1 1s 1s/step
1/1 1s 916ms/step
1/1 1s 871ms/step
1/1 1s 899ms/step
1/1 1s 1s/step
1/1 1s 880ms/step
Confusion Matrix
Glioma
Meningioma
No Tumor
Pituitary
True
121 12 0 3
3 114 1 22
3 7 85 5
1 0 1 134
120
100
80
60
40
20
0
Glioma Meningioma No Tumor Pituitary
Predicted
25°C Sunny
Search
ENG IN
13:45 12-12-2025